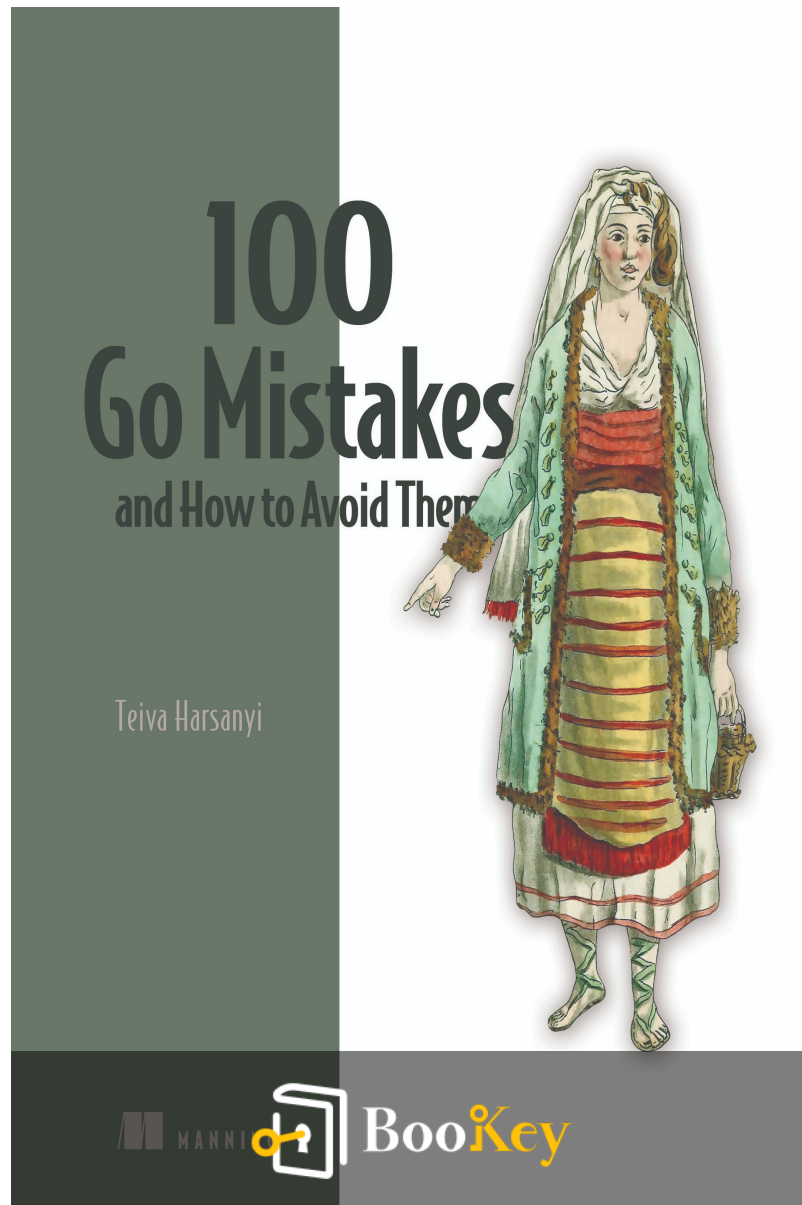


100 Go Mistakes and How to Avoid Them PDF

Teiva Harsanyi



More Free Book



Scan to Download



Listen It

100 Go Mistakes and How to Avoid Them

Master Go: Avoid Mistakes and Enhance Your
Programming Skills

Written by Bookey

[Check more about 100 Go Mistakes and How to Avoid Them Summary](#)

[Listen 100 Go Mistakes and How to Avoid Them Audiobook](#)

More Free Book



Scan to Download



[Listen It](#)

About the book

"100 Go Mistakes and How to Avoid Them" is an essential guide for developers looking to enhance their Go programming skills by identifying and correcting common errors that even seasoned professionals make. This insightful book offers a wealth of techniques for crafting idiomatic, expressive, and efficient Go code, while providing valuable insights through engaging examples and real-world case studies. Focused exclusively on pure Go code, it presents universally applicable standards that can elevate any project. Readers will delve into the nuances of handling JSON data and HTTP services, master best practices for code organization, and optimize slice usage. By improving concurrency, adeptly managing errors, and refining testing quality, you'll significantly boost both the speed and reliability of your Go applications.

More Free Book



Scan to Download



[Listen It](#)

About the author

Teiva Harsanyi is a seasoned entrepreneur and business strategist with a wealth of experience in navigating the complexities of the financial landscape. Known for his keen insights and practical advice, Harsanyi has dedicated his career to helping individuals and organizations avoid common pitfalls in their financial journeys. With a background in both finance and strategic planning, he combines theory with real-world applications, making his teachings accessible and actionable. In "100 Go Mistakes and How to Avoid Them," Harsanyi distills his extensive knowledge into actionable lessons, empowering readers to identify and circumvent critical missteps in their ventures, ultimately paving the way for success.

More Free Book



Scan to Download



[Listen It](#)

Ad



Scan to Download



Try Bookey App to read 1000+ summary of world best books

Unlock **1000+** Titles, **80+** Topics

New titles added every week

Brand



Leadership & Collaboration



Time Management



Relationship & Communication



Business Strategy



Creativity



Public



Money & Investing



Know Yourself



Positive Psychology

Entrepreneurship



World History



Parent-Child Communication

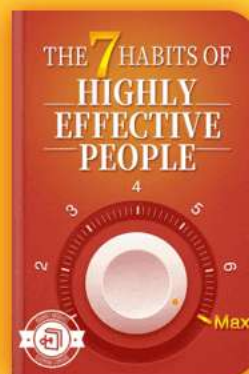
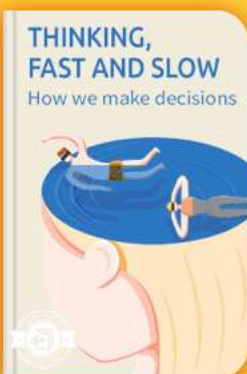


Self-care



Mind & Spirituality

Insights of world best books



Free Trial with Bookey



Summary Content List

Chapter 1 : 100 Go Mistakes

Chapter 2 : 1 Go: Simple to learn but hard to master

Chapter 3 : 2 Code and project organization

Chapter 4 : 3 Data types

Chapter 5 : 4 Control structures

Chapter 6 : 5 Strings

Chapter 7 : 6 Functions and methods

Chapter 8 : 7 Error management

Chapter 9 : 8 Concurrency: Foundations

Chapter 10 : 9 Concurrency: Practice

Chapter 11 : 10 The standard library

Chapter 12 : 11 Testing

Chapter 13 : 12 Optimizations

More Free Book



Scan to Download



Listen It

Chapter 1 Summary : 100 Go Mistakes



Summary of Chapter 1 from "100 Go Mistakes and How to Avoid Them"

Preface

In 2019, Teiva Harsanyi began his journey with Go as the primary language and noticed recurrent coding mistakes. Writing a blog post titled “The Top 10 Most Common Mistakes I’ve Seen in Go Projects” garnered significant attention, motivating him to document 100 common Go mistakes. After extensive research and collaboration, he partnered with Manning to publish this book, aimed at helping developers improve their Go proficiency by avoiding

More Free Book



Scan to Download



Listen It

these mistakes.

Acknowledgments

Harsanyi expresses gratitude to his family, mentors, and the Go community for their support and feedback, which enhanced the quality of the book.

About This Book

The book outlines 100 common mistakes developers encounter in Go, focusing on core aspects of the language and accompanied by practical examples. It doesn't cater to beginners but is intended for those with some experience in Go projects.

Book Organization

The book is structured into 12 chapters, covering topics including:

1.

Go: Simple to Learn but Hard to Master

- Discusses the complexities of mastering Go.

2.

More Free Book



Scan to Download



Listen It

Code and Project Organization

- Addresses mistakes in codebase structure.

3.

Data Types

- Covers issues with basic types, slices, and maps.

4.

Control Structures

- Explores common control structure errors.

5.

Strings

- Looks at string representation mistakes.

6.

Functions and Methods

- Discusses function-related errors.

7.

Error Management

- Covers idiomatic error handling.

8.

Concurrency: Foundations

- Introduces concurrency concepts.

9.

Concurrency: Practice

- Illustrates mistakes in concurrency application.

10.

More Free Book



Scan to Download



Listen It

The Standard Library

- Examines common misuses of the standard library.

11.

Testing

- Identifies mistakes in testing and benchmarking.

12.

Optimizations

- Explores application performance optimization techniques.

Code Annotations

The book includes various examples of code formatted distinctly to highlight changes, providing a clear understanding of concepts. Additional resources are available online for readers to access executable code snippets and further discussions.

Author Information

Teiva Harsanyi, a senior software engineer at Docker, shares his insights from working across diverse industries and his passion for Go and application reliability.

Cover Illustration

More Free Book



Scan to Download



Listen It

The cover features an illustration from 1797, representing regional culture and celebrating the diversity of historical perspectives in the computer industry.

More Free Book



Scan to Download



Listen It

Critical Thinking

Key Point: The potential pitfalls in mastering Go programming.

Critical Interpretation: Harsanyi posits that while Go is designed to be user-friendly, the mastery of its intricacies presents challenges that experienced developers often overlook. This assertion raises questions about the universality of his perspective, as not all developers encounter the same issues. For those who seek a broader understanding, sources like 'The Pragmatic Programmer' by Andrew Hunt and David Thomas emphasize varying experiences in skill acquisition across programming languages, which may suggest that his observations do not encompass the full range of Go developers' experiences.

More Free Book



Scan to Download



Listen It

Chapter 2 Summary : 1 Go: Simple to learn but hard to master



1 Go: Simple to Learn but Hard to Master

This chapter covers the efficiency, scalability, and productivity of the Go programming language, highlighting the balance between its simplicity for learning and the complexities in mastering its nuances. It discusses the importance of learning from mistakes, a key idea to becoming proficient in Go, by exploring 100 common mistakes made by developers.

1.1 Go Outline

More Free Book



Scan to Download



Listen It

This section highlights key aspects of Go that make it a powerful language, including:

-

Stability:

Consistent updates while maintaining stability.

-

Expressivity:

Simple syntax promotes natural and intuitive coding.

-

Compilation:

Fast compilation times enhance productivity.

-

Safety:

Strong, static typing ensures code safety.

Despite its straightforward learning curve, mastering Go requires an understanding of intricate concepts, particularly concurrency, which demonstrates that “simple doesn’t mean easy”.

1.2 Simple Doesn’t Mean Easy

This section differentiates between "simple" and "easy." While Go's main features can be learned quickly, mastering the language involves overcoming challenges like

More Free Book



Scan to Download



Listen It

understanding concurrency complexities. The observation that even simple concepts, such as channels and goroutines, can lead to significant bugs exemplifies the need for deeper comprehension.

1.3 100 Go Mistakes

The importance of learning from common mistakes is emphasized using neuroscience findings that suggest errors aid in deeper learning. This book categorizes mistakes into seven main groups:

-

1.3.1 Bugs:

Software bugs, leading to significant costs and tragic consequences, need to be avoided.

-

1.3.2 Needless Complexity:

Avoid unnecessary design for hypothetical future needs, which complicates codebases.

-

1.3.3 Weaker Readability:

Developing readable code is crucial for long-term maintenance.

-

More Free Book



Scan to Download



Listen It

1.3.4 Suboptimal or Unidiomatic Organization:

Proper project organization aids understandability and maintainability.

-

1.3.5 Lack of API Convenience:

User-friendly APIs enhance usability, requiring careful design choices.

-

1.3.6 Under-Optimized Code:

Performance and accuracy could be compromised by not understanding language features.

-

1.3.7 Lack of Productivity:

Maximizing productivity through comfort and effective use of language features is essential.

The chapter concludes with a summary emphasizing Go's potential for enhancing developer productivity while acknowledging the necessary journey of learning through understanding mistakes.

More Free Book



Scan to Download



Listen It

Example

Key Point: Learning from mistakes is essential in mastering Go's concepts, especially concurrency.

Example: Imagine diving into your first Go project. You write a piece of code that elegantly utilizes goroutines to handle concurrent tasks. At first glance, everything works seamlessly. However, as usage increases, unforeseen race conditions emerge, causing unexpected crashes. This experience teaches you the pivotal lesson of addressing these concurrency issues, highlighting that grasping the nuances of Go's communication patterns isn't just about following examples—it's about deeply understanding how they function in real-time. As you iteratively debug and refine your approach, you realize that every mistake brings invaluable insight, helping you evolve into a proficient Go developer.

More Free Book



Scan to Download



Listen It

Critical Thinking

Key Point: Mastering Go requires deep understanding of its complexities despite its simple learning curve.

Critical Interpretation: While Teiva Harsanyi argues that the Go programming language is simple to learn yet difficult to master, one must critically understand that simplifying programming concepts may lead to superficial comprehension. Many developers might find initial success with Go but struggle with more intricate aspects like concurrency. This suggests that the author's assertion about simplicity could overshadow the true challenges developers face, leading to a false sense of security. Understanding nuances in programming languages, particularly Go, is essential for avoiding serious bugs and ensuring code quality (Pérez et al., 2019; Bird et al., 2009). Readers might explore various resources and publications to gain a multi-faceted understanding of Go beyond the author's perspective.

More Free Book



Scan to Download



Listen It

Chapter 3 Summary : 2 Code and project organization

Mistake	Description
Unintended Variable Shadowing	Redeclaring a variable in an inner block can lead to unintended behavior. Use temporary variables or direct assignments to prevent this.
Unnecessary Nested Code	Deeply nested structures reduce readability. Simplify code by eliminating extra else blocks and focusing on the happy path.
Misusing Init Functions	Init functions can complicate error management and hinder testing. Prefer conventional function initializations instead.
Overusing Getters and Setters	Excessive use of these does not conform to idiomatic Go practices. Use them only when needed for additional functionalities.
Interface Pollution	Creating too many interfaces adds complexity. Design interfaces based on actual needs, not anticipated future requirements.
Interface on the Producer Side	Defining interfaces with implementations limits client flexibility. Create interfaces from the client's context for relevance.
Returning Interfaces	Using interfaces over concrete types complicates dependencies. Prefer returning structs to maintain flexibility.
Any Says Nothing	The empty interface (<code>`any`</code>) reduces type clarity. Be explicit with types unless necessary to use <code>`any`</code> .
Being Confused About Generics	Generics should enhance simplicity, not be a default choice. Use them judiciously to avoid complexity.
Problems with Type Embedding	Type embedding should promote method exposure without revealing internal fields, to maintain encapsulation.
Not Using Functional Options Pattern	This pattern aids in managing optional settings in APIs, allowing flexible configuration without complicating function signatures.
Project Misorganization	A well-structured project promotes maintainability. Follow a consistent layout and naming conventions.
Creating Utility Packages	Generic utility packages reduce clarity. Create specific packages for better understanding of their functionality.
Ignoring Package Name Collisions	Name collisions cause confusion. Avoid identical names for packages and variables or use aliases to avoid issues.
Missing Code Documentation	Documentation for exported elements enhances usability and maintainability, providing clear guidance.
Not Using Linters	Linters help detect potential errors. Regular use enhances code quality and consistency.

More Free Book



Scan to Download



Listen It

Code and Project Organization

This chapter focuses on organizing Go code to ensure it remains clean, idiomatic, and maintainable. It discusses common mistakes and best practices related to project structure, interface and generics usage, initializations, as well as utility packages.

1. Unintended Variable Shadowing

Variable shadowing occurs when a variable name is redeclared in an inner block. This can lead to unintended behavior where the outer variable remains unassigned. Proper variable assignment can be achieved by using temporary variables or direct assignments to prevent shadowing errors.

2. Unnecessary Nested Code

Install Bookey App to Unlock Full Text and Audio

More Free Book



Scan to Download



Listen It



Scan to Download



Why Bookey is must have App for Book Lovers



30min Content

The deeper and clearer interpretation we provide, the better grasp of each title you have.



Text and Audio format

Absorb knowledge even in fragmented time.



Quiz

Check whether you have mastered what you just learned.



And more

Multiple Voices & fonts, Mind Map, Quotes, IdeaClips...

Free Trial with Bookey



Chapter 4 Summary : 3 Data types

3 Data Types

This chapter discusses common mistakes related to data types, specifically focusing on basic types, slices, and maps in Go.

3.1 #17: Creating Confusion with Octal Literals

Octal integer literals in Go, indicated by a leading zero (0), can lead to confusion. For example, ``010`` is treated as base 8 (octal), equaling 8 in base 10, so ``sum := 100 + 010`` results in 108, not 110. To improve code clarity, use the ``0o`` prefix for octal numbers (e.g., ``0o644`` for file permissions). It's advised to keep in mind the various integer representations: binary (``0b``), hexadecimal (``0x``), and also how to improve readability with underscores (e.g., ``1_000_000``).

3.2 #18: Neglecting Integer Overflows

Integer overflows in Go are silent, leading to potential bugs. Go features several integer types and provides no runtime

More Free Book



Scan to Download



Listen It

panic for overflows, though detectable during compile time. To prevent issues, maintain checks against ``math.MaxInt`` and handle potential overflows for operations, increments, and multiplications.

3.3 #19: Not Understanding Floating Points

Go offers ``float32`` and ``float64`` for floating-point arithmetic, which is an approximation of real values. One should be cautious with floating-point calculations as they can result in decreased precision, depending on the CPU architecture. For comparisons, always use a delta range instead of direct equality checks. Additionally, Go has special floating-point numbers like NaN and Infinity.

3.4 #20: Not Understanding Slice Length and Capacity

Developers must comprehend the difference between slice length (number of elements) and capacity (allocated storage). Misunderstanding this can lead to inefficient use of memory, such as allocating unnecessary space or risking index out-of-range errors.

More Free Book



Scan to Download



Listen It

3.5 #21: Inefficient Slice Initialization

Properly initializing slices with the correct capacity or length avoids performance overhead due to repeated allocations. Allocate slices with ``make([]Type, length, capacity)`` to optimize performance and avoid memory issues.

3.6 #22: Being Confused About Nil vs. Empty Slices

Nil slices (``var s []string``) require no allocation, while empty slices (``make([]string, 0)``) do. Understanding when to use either is essential to preventing unnecessary memory usage.

3.7 #23: Not Properly Checking if a Slice is Empty

Use ``len(slice) == 0`` as the idiomatic way to check for empty slices, as comparing nil slices does not always consider empty slices.

3.8 #24: Not Making Slice Copies Correctly

The built-in ``copy`` function requires proper handling of destination slices to avoid zero-length issues. A destination slice must have sufficient length to store copied elements.



3.9 #25: Unexpected Side Effects Using Slice Append

Be mindful that appending to a slice can affect all slices that share the same backing array. Use slice copies to prevent unintended modifications.

3.10 #26: Slices and Memory Leaks

Slicing a larger slice can retain unused capacity, potentially leading to high memory usage. Always prefer using slice copies to mitigate these leaks.

3.11 #27: Inefficient Map Initialization

Maps should be initialized with a size hint when the number of elements is known, which optimizes memory allocation and reduces the cost of resizing during runtime.

3.12 #28: Maps and Memory Leaks

Go's maps cannot shrink in size. Even after clearing elements, memory used for buckets remains allocated. Consider recreating maps to reduce memory consumption if

More Free Book



Scan to Download



Listen It

necessary.

3.13 #29: Comparing Values Incorrectly

The `==` operator can't be used for all types (e.g., slices and maps). Use methods like `reflect.DeepEqual` for complex types or implement custom comparison functions as needed.

Summary

In conclusion, throughout the chapter, recognizing the correct use of octal literals, handling integer overflows, understanding floating-point precision, and managing slices and maps efficiently is critical for Go developers. By adhering to these principles, one can avoid common pitfalls and write more robust code.

More Free Book



Scan to Download



Listen It

Critical Thinking

Key Point: Understanding the nuances of Go's data types is crucial in avoiding common pitfalls.

Critical Interpretation: In this chapter, Harsanyi emphasizes the complexity surrounding fundamental data types in Go, particularly regarding octal literals, integer overflows, and slice management. However, while his focus on these topics encourages better coding practices, one must consider that this perspective may oversimplify the broader context of programming language principles. Other sources, such as 'Effective Go' and 'Programming in Go' by Brian Kernighan, offer a more holistic view of language design and functionality, suggesting the need for developers to adapt to varied language idiosyncrasies rather than strictly adhering to prescribed practices.

More Free Book



Scan to Download



Listen It

Chapter 5 Summary : 4 Control structures

4 Control Structures

This chapter covers various aspects of control structures in Go, particularly focusing on range loops and common pitfalls associated with them. Unlike C or Java, Go only has a generalized for loop, which can cause misunderstandings, especially with how values are assigned during iteration.

4.1 Range Loops

4.1.1 Concepts

A range loop simplifies iterating over data structures without manually handling indexes. It produces pairs of index and element values for most structures, while the blank identifier can be used to ignore the index.

4.1.2 Value Copy

More Free Book



Scan to Download



Listen It

In range loops, each element is copied to a value variable. If a struct is updated within a range loop, only the copy is modified, not the original slice element. To mutate original elements, access them via index or use a slice of pointers.

4.2 Evaluating Arguments in Range Loops

4.2.1 Channels

The expression in a range loop is evaluated once before the loop starts. This means iterating over a channel doesn't adjust based on changes to the channel within the loop.

4.2.2 Arrays

When using range with arrays, the loop operates on a copy of the array. Modifications to the original array during the iteration do not affect the value variable produced by range.

4.3 Pointer Elements in Range Loops

Using pointers can lead to common mistakes, such as

More Free Book



Scan to Download



Listen It

unintentionally referencing the same instance in a loop. It's crucial to create a local variable or use the slice index to prevent multiple references to the same pointer.

4.4 Map Iteration Issues

4.4.1 Ordering

Maps do not maintain order or provide deterministic iteration. Always avoid making assumptions about the order when iterating over maps.

4.4.2 Map Updates During Iteration

While modifying a map during iteration is allowed, it can yield unpredictable results. To reliably control modifications, work on a copy of the map.

4.5 Break Statement Misunderstandings

A break statement will only terminate the innermost control structure (for, switch, select). To break the outer loop, use labels.

More Free Book



Scan to Download



Listen It

4.6 Defer Inside Loops

Using defer inside loops delays function calls until the enclosing function returns. To avoid keeping resources open indefinitely, extract the loop logic into its own function.

Summary

Understanding the mechanics of range loops, expression evaluation, pointer references, map behavior, break statements, and the usage of defer helps avoid common pitfalls in Go programming. Use local variables and indexes effectively, avoid assumptions about maps, and ensure that cleanup occurs properly when using defer.

More Free Book



Scan to Download



Listen It

Chapter 6 Summary : 5 Strings

Section	Key Points
Chapter 6: Strings in Go	This chapter focuses on understanding string handling in Go, discussing common mistakes, optimization strategies, and memory management.
5.1: Understanding the Concept of a Rune	A rune represents a Unicode code point essential for string operations. Distinction between charset (characters) and encoding (binary representation). Go strings are immutable byte sequences and UTF-8 encoded. Length returns the number of bytes, not characters.
5.2: Inaccurate String Iteration	Confusion can arise in string iteration, especially with multi-byte runes. Incorrect use of range prints bytes, not runes. Use values from range loop or convert to a slice of runes for accurate runes output.
5.3: Misusing Trim Functions	Common errors include confusing <code>TrimRight</code> (trailing) with <code>TrimSuffix</code> (exact suffix). Understanding functions like <code>TrimLeft</code>, <code>TrimPrefix</code>, and <code>Trim</code> ensures correct string manipulation.
5.4: Under-optimized String Concatenation	Using the <code>+=</code> operator for string concatenation leads to performance issues. <code>strings.Builder</code> is recommended for efficient concatenation. Preallocating space with <code>Grow</code> can enhance performance in high-volume situations.
5.5: Useless String Conversions	Favoring strings over <code>[]byte</code> can lead to inefficiencies, especially in I/O operations. Use the <code>bytes</code> package for direct byte slice manipulation to avoid unnecessary conversions.
5.6: Substrings and Memory Leaks	Substring operations may cause memory leaks due to shared backing arrays. Create distinct copies of substrings for significant memory management. As of Go 1.18, use <code>strings.Clone</code> for safer string copying.
Summary	

More Free Book



Scan to Download



Listen It

Section	Key Points
	Understanding runes and byte manipulation is essential in Go. Proper iteration and function usage help avoid common pitfalls. Optimization uses <code>strings.Builder</code> and <code>bytes</code> package to prevent memory issues.

Chapter 6: Strings in Go

5 Strings

This chapter focuses on understanding string handling in Go, discussing common mistakes, optimization strategies, and memory management.

5.1: Understanding the Concept of a Rune

- A rune in Go represents a Unicode code point and is crucial for string operations.
- Distinction between charset (set of characters) and encoding (binary representation).
- Go strings reference immutable byte sequences, and all string literals are UTF-8 encoded.
- Length of a string returns the number of bytes, not characters.

More Free Book



Scan to Download



Listen It

5.2: Inaccurate String Iteration

- Iterating strings can lead to confusion, especially with multi-byte runes.
- Example highlights incorrect iteration output; using the range operator incorrectly results in printed bytes instead of runes.
- To print runes accurately, use the value from the range loop or convert to a slice of runes.

5.3: Misusing Trim Functions

- Common errors include confusing ``TrimRight`` (removes trailing characters) and ``TrimSuffix`` (removes exact suffix).
- Understanding the differences ensures correct string manipulation using ``TrimLeft``, ``TrimPrefix``, and ``Trim``.

Install Bookey App to Unlock Full Text and Audio

More Free Book



Scan to Download



Listen It

Ad



Scan to Download



App Store
Editors' Choice



22k 5 star review

Positive feedback

Sara Scholz

...tes after each book summary
...erstanding but also make the
...and engaging. Bookey has
...ding for me.

Fantastic!!!



I'm amazed by the variety of books and languages
Bookey supports. It's not just an app, it's a gateway
to global knowledge. Plus, earning points for charity
is a big plus!

Masood El Toure

Fi



Ab
bo
to
my

José Botín

...ding habit
...o's design
...ual growth

Love it!



Bookey offers me time to go through the
important parts of a book. It also gives me enough
idea whether or not I should purchase the whole
book version or not! It is easy to use!

Wonnie Tappkx

Time saver!



Bookey is my go-to app for
summaries are concise, ins
curated. It's like having acc
right at my fingertips!

Awesome app!



I love audiobooks but don't always have time to listen
to the entire book! bookey allows me to get a summary
of the highlights of the book I'm interested in!!! What a
great concept !!!highly recommended!

Rahul Malviya

Beautiful App



This app is a lifesaver for book lovers with
busy schedules. The summaries are spot
on, and the mind maps help reinforce wh
I've learned. Highly recommend!

Alex Walk

Free Trial with Bookey



Chapter 7 Summary : 6 Functions and methods

6 Functions and Methods This Chapter Covers

This chapter discusses the following key concepts related to Go functions and methods:

- Choosing between value or pointer receivers.
- Using named result parameters and their implications.
- Avoiding common mistakes such as returning a nil receiver.
- Problems associated with functions that accept filenames.
- Handling defer arguments correctly.

6.1 Not Knowing Which Type of Receiver to Use

Choosing between value and pointer receivers is crucial in Go.

-

Value Receiver:

A copy of the value is passed, so changes do not affect the original object.

More Free Book



Scan to Download



Listen It

-

Pointer Receiver:

The address is passed, allowing modifications to affect the original object.

Guiding principles for choosing receivers:

- Use pointer receivers if mutating the receiver or if the receiver is large.
- Use value receivers if enforcing immutability or when working with small, simple types like basic data types and structs without mutable fields.

Note on mixing receiver types: Generally avoided, but permissible in certain cases from the standard library.

6.2 Never Using Named Result Parameters

Named result parameters provide an option to name function results for clarity. They can streamline function signatures but should be used judiciously.

Best practices:

- Use in interface definitions to clarify meaning.
- Avoid in cases where names do not enhance readability or

More Free Book



Scan to Download



Listen It

carry fewer benefits, like when returned variables are of similar types.

***Warnings*:** Named parameters can lead to confusion if not handled carefully due to them being initialized to zero.

6.3 Unintended Side Effects with Named Result Parameters

Care must be taken when using named result parameters. If these parameters are not correctly assigned, they can lead to subtle bugs where a variable returns its zero value rather than an intended one.

***Example*:** Returning a nil error when it should convey meaningful failure information.

Proposed solutions involve proper assignment before return and ensuring clarity in return statements.

6.4 Returning a Nil Receiver

Returning a nil receiver (like a nil pointer) can cause erroneous behavior as the caller may receive a non-nil interface.

More Free Book



Scan to Download



Listen It

***Solution*:** Only return a receiver if it's not nil; otherwise, return nil explicitly to ensure clarity.

6.5 Using a Filename as a Function Input

Accepting a filename as input can hinder testing and reduce code reusability.

***Better approach*:** Accept an `io.Reader` to abstract the input source, allowing flexibility in how the function is reused and easier unit testing.

6.6 Ignoring How Defer Arguments and Receivers Are Evaluated

Understanding how defer works is essential:

-

Argument Evaluation

: Defer statements evaluate their arguments immediately, which can lead to unexpected results if variables in the function change afterward.

***Solutions*:** Use pointers or closures to maintain intended



behavior.

-

Receiver Evaluation

: Like arguments, the method receivers are evaluated immediately. The visibility of mutations depends on whether the receiver is a pointer or a value.

Summary

- Choose receiver types wisely based on the requirements.
- Use named result parameters to enhance readability but be cautious of potential pitfalls.
- Avoid returning nil pointers when dealing with interfaces to prevent unintended errors.
- Prefer using `io.Reader` for functions requiring input from files to enhance testability and reusability.
- Be aware of how `defer` evaluates its arguments to prevent issues in your code.

More Free Book



Scan to Download



Listen It

Chapter 8 Summary : 7 Error management

Error Management in Go

This chapter covers key aspects of error management in Go applications, emphasizing that it is as crucial as other parts of a codebase. In Go, errors are treated as regular return values rather than using traditional try/catch mechanisms like many other programming languages.

Panicking

Panic is a built-in function in Go that interrupts the normal flow of a program. It is appropriate to use panic for genuinely exceptional conditions, such as programmer errors or important dependency failures. However, it should be used sparingly.

Ignoring When to Wrap an Error

Since Go 1.13, developers can wrap errors using the %w

More Free Book



Scan to Download



Listen It

directive to provide additional context or mark the error as a specific type. The chapter explains appropriate situations for wrapping errors, such as adding context around errors or indicating specific error types.

Checking an Error Type Inaccurately

When using error wrapping, developers must use ``errors.As`` for checking specific error types rather than type assertion. This allows for correctly handling errors even if they are wrapped.

Checking an Error Value Inaccurately

Sentinel errors are used to represent expected errors. This section emphasizes using ``errors.Is`` to compare an error against a specific value, particularly if the error may be wrapped during processing.

Handling an Error Twice

It's a common mistake to handle the same error multiple times, which can complicate debugging. Developers are encouraged to either log or return an error—never both—to

More Free Book



Scan to Download



Listen It

maintain clarity in error management.

Not Handling an Error

When intentionally ignoring an error, it is essential to do so explicitly by assigning it to the blank identifier. This clarifies the developer's intention for maintainability and avoids confusion.

Not Handling Defer Errors

Errors returned from deferred function calls should not be neglected. Developers can either handle these errors by logging them or propagate them back to the caller with appropriate context.

Summary

Panic should be reserved for exceptional states. Error wrapping adds context but creates coupling, and it is vital to comprehend how to check types and values correctly when errors are wrapped. Errors need to be handled only once, either logged or returned, and ignored errors must be made explicit to avoid confusion. Error management is crucial for creating robust and maintainable Go applications.

More Free Book



Scan to Download



Listen It

Critical Thinking

Key Point:Error Management in Go is Essential to Code Quality

Critical Interpretation:The chapter argues that error management is fundamental to Go applications, presenting best practices for handling errors effectively. While the author's emphasis on error management is valid and aligns with best practices in software engineering, it is also essential to recognize that different programming paradigms may handle errors in varied ways, and Go's approach may not suit every development scenario. The notion that errors should be treated as first-class values is prevalent in Go, yet critics may argue that traditional try/catch mechanisms could provide clearer separation of error-handling logic, as seen in languages like Java or Python (see 'Clean Code' by Robert C. Martin). Thus, developers are urged to critically evaluate whether Go's model aligns with their project's needs.

More Free Book



Scan to Download



Listen It

Chapter 9 Summary : 8 Concurrency: Foundations

Concurrency: Foundations

This chapter explores the essentials of concurrency in Go, covering principles that guide developers in writing efficient concurrent code. It distinguishes between concurrency and parallelism, addresses misconceptions about performance, discusses synchronization mechanisms, and explains how to work with Go contexts.

Understanding Concurrency and Parallelism

Concurrency and parallelism are often confused by developers. The chapter uses a coffee shop analogy to illustrate the distinction:

-

Concurrency

is about structuring a program to deal with multiple tasks at once, enabling independent processes to communicate and synchronize.

More Free Book



Scan to Download



Listen It

-

Parallelism

entails executing multiple tasks simultaneously, enhancing performance by utilizing additional resources like cores. The chapter emphasizes that concurrency is necessary for leveraging modern multi-core CPUs, though it does not always guarantee faster execution.

Thinking Concurrency is Always Faster

A common misconception is that concurrent solutions automatically outperform sequential ones. Performance depends on various factors, including task structure and contention among workloads. The chapter stresses:

- Goroutines (Go's lightweight threads) and how they're managed by the Go scheduler.
- The importance of considering workload types—CPU-bound versus I/O-bound—when implementing

Install Bookey App to Unlock Full Text and Audio

More Free Book



Scan to Download



Listen It



Read, Share, Empower

Finish Your Reading Challenge, Donate Books to African Children.

The Concept



×



×



This book donation activity is rolling out together with Books For Africa. We release this project because we share the same belief as BFA: For many children in Africa, the gift of books truly is a gift of hope.

The Rule



Earn 100 points



Redeem a book



Donate to Africa

Your learning not only brings knowledge but also allows you to earn points for charitable causes! For every 100 points you earn, a book will be donated to Africa.

Free Trial with Bookey



Chapter 10 Summary : 9 Concurrency: Practice

Section	Common Mistake	Guidance
9.1	Propagating an Inappropriate Context	Use an empty or custom context to avoid unexpected cancellation.
9.2	Starting a Goroutine Without Knowing When to Stop	Ensure goroutines can exit cleanly by using cancellation contexts.
9.3	Not Being Careful with Goroutines and Loop Variables	Create new loop variables for each goroutine or use functions with parameters.
9.4	Expecting Deterministic Behavior Using Select and Channels	Prepare for non-deterministic outcomes by handling all select cases.
9.5	Not Using Notification Channels	Use <code>`chan struct{}`</code> for non-data notifications for clarity.
9.6	Not Using Nil Channels	Use nil channels to manage flow and prevent busy waits.
9.7	Being Puzzled About Channel Size	Use buffered or unbuffered channels appropriately, starting with size 1.
9.8	Forgetting About Possible Side Effects with String Formatting	Avoid unintentional data races or deadlocks with careful string formatting.
9.9	Creating Data Races with Append	Manage concurrent modifications on slices to avoid data races.
9.10	Using Mutexes Inaccurately with Slices and Maps	Locking mechanisms must be precise; copy data explicitly when needed.
9.11	Misusing <code>sync.WaitGroup</code>	Call <code>`Add`</code> before launching goroutines to ensure proper counting.
9.12	Forgetting About <code>sync.Cond</code>	Use <code>`sync.Cond`</code> to avoid busy-waiting and signal multiple goroutines efficiently.
9.13	Not Using <code>errgroup</code>	Utilize <code>`errgroup`</code> to simplify concurrency and error handling.
9.14	Copying a Sync Type	Avoid copying sync types; use pointers to mitigate data races.
Summary	Understanding context, goroutine lifecycle, channels, and synchronization is essential for efficient Go applications. Utilize idiomatic practices and packages like <code>`errgroup`</code> .	

More Free Book



Scan to Download



Listen It

Concurrency: Practice

This chapter addresses common mistakes made by Go developers related to concurrency and offers guidance on avoiding these pitfalls. The focus is on using goroutines, channels, and managing concurrency with synchronization primitives effectively.

9.1 Propagating an Inappropriate Context

Context propagation is crucial but can lead to subtle bugs when the original context becomes canceled unexpectedly, particularly after an HTTP response is sent. Solutions include using an empty context or creating a custom context that does not carry cancellation signals while retaining necessary values.

9.2 Starting a Goroutine Without Knowing When to Stop

Starting goroutines without a plan to stop them can lead to memory leaks and resource exhaustion. It's essential to ensure goroutines can exit cleanly, often by utilizing contexts

More Free Book



Scan to Download



Listen It

that trigger cancellation or by properly managing resource cleanup.

9.3 Not Being Careful with Goroutines and Loop Variables

Goroutines can mistakenly reference loop variables, leading to unpredictable behavior. This can be resolved by creating a new variable within the loop for each goroutine or refactoring to use functions that accept parameters.

9.4 Expecting Deterministic Behavior Using Select and Channels

Go's select statement may choose cases randomly when multiple channels can proceed, which can yield non-deterministic results. Developers should not assume the first case will always be selected and should be prepared for such behavior by implementing proper handling for all cases.

9.5 Not Using Notification Channels

For notifications that don't need to convey data, such as signaling events, Go developers should use `chan struct{}`

More Free Book



Scan to Download



Listen It

for clarity and efficiency.

9.6 Not Using Nil Channels

Nil channels can block forever, and their behavior can be intentionally utilized to manage flow in concurrent applications. Properly using nil channels can simplify complexity and prevent busy waits.

9.7 Being Puzzled About Channel Size

Understanding when to use buffered vs. unbuffered channels, and choosing an appropriate size within context (usually starting with size 1), can help prevent deadlocks and contention issues.

9.8 Forgetting About Possible Side Effects with String Formatting

Developers must be cautious with string formatting in concurrent applications, as it can lead to data races or deadlocks due to unintentional calls to methods.

9.9 Creating Data Races with Append

More Free Book



Scan to Download



Listen It

Using append on slices in a concurrent context can lead to data races. If slices are not full, concurrent modifications need to be managed carefully to avoid race conditions.

9.10 Using Mutexes Inaccurately with Slices and Maps

Using mutexes to protect shared data structures requires precision. Simply copying maps or slices can lead to undetected data races, necessitating proper locking mechanisms across the entire function or copying data explicitly.

9.11 Misusing sync.WaitGroup

When using `sync.WaitGroup`, the `Add` method should always be called before launching goroutines to ensure proper counting; otherwise, behavior becomes nondeterministic.

9.12 Forgetting About sync.Cond

Utilizing `sync.Cond` allows for more efficient signaling

More Free Book



Scan to Download



Listen It

between goroutines. It prevents busy-waiting and can signal multiple waiting goroutines efficiently.

9.13 Not Using `errgroup`

The ``errgroup`` package simplifies concurrent operations and error handling in Go applications, providing a clean way to manage multiple goroutines without requiring complex structures.

9.14 Copying a Sync Type

Sync types such as mutexes should never be copied as this leads to data races. Utilizing pointers to sync types can mitigate this issue.

Summary

Understanding context management, goroutine lifecycle, channel behavior, and proper use of synchronization primitives are vital for writing efficient, race-free concurrent Go applications. Developers are encouraged to leverage Go's idiomatic practices and available packages like ``errgroup`` to streamline their concurrency models.

More Free Book



Scan to Download



Listen It

Chapter 11 Summary : 10 The standard library

Chapter 11: The Standard Library

This chapter discusses common mistakes made when using the Go standard library, emphasizing the importance of understanding its features and behaviors to avoid bugs in production-grade applications.

Common Mistakes in Time Handling

-

Providing a Wrong Time Duration

: Many developers confuse the `time.Duration`` type and mistakenly provide durations in milliseconds instead of nanoseconds. Always use the `time.Duration`` API to avoid this pitfall.

-

Memory Leaks with `time.After``

: Using `time.After`` in loops can cause memory leaks since the resources are only released when the timer expires.

More Free Book



Scan to Download



Listen It

Instead, utilize `time.NewTimer` for better memory management.

JSON Handling Mistakes

-

Unexpected Behavior from Type Embedding

: When embedding types, such as `time.Time`, the marshalling can behave unexpectedly, resulting in missing fields. Changes to default JSON marshaling can lead to bugs.

-

Monotonic Clock Issues

: The `time.Time` type includes both wall and monotonic clocks, causing unexpected comparison results. Use the `Equal` method or `Truncate` to compare without considering the monotonic part.

-

Maps of `any`

: Unmarshalling into maps yields all numeric values as `float64`, leading to potential errors if developers expect them to be integers.

Common SQL Handling Mistakes

More Free Book



Scan to Download



Listen It

-

Misunderstanding `sql.Open`

: This function does not establish a database connection; use `Ping` to confirm connection availability.

-

Connection Pooling

: Understand how connection pooling works to avoid exhausting resources. Adjust parameters for production use based on expected load.

-

Prepared Statements

: Use prepared statements to optimize performance and enhance security against SQL injection.

-

Handling Null Values

: Use pointer types or `sql.NullXXX` types to prevent errors when scanning null database values.

-

Row Iteration Errors

: Always check for errors after iterating over rows to ensure proper error handling.

Resource Management

More Free Book



Scan to Download



Listen It

-

Not Closing Transient Resources

:

- Always close HTTP response bodies to prevent resource leaks.

- Ensure that ``sql.Rows`` and ``os.File`` are closed to prevent connection and file descriptor leaks.

HTTP Handling Mistakes

-

Missing Return Statements

: Forgetting to return after handling an error in an HTTP handler can lead to unintended behaviors.

-

Using Default HTTP Client and Server

: The default implementations lack essential timeout settings that are critical in production environments. It's important to customize these settings for robust and secure applications.

Summary

Developers must remain cautious with ``time.Duration``, be aware of memory management when using timers, avoid

More Free Book



Scan to Download



Listen It

common JSON pitfalls, and ensure proper database interaction. Proper resource management and explicit error handling in HTTP handlers are also critical for creating reliable applications. Always customize default implementations in production to enhance performance and security.

More Free Book



Scan to Download



Listen It

Chapter 12 Summary : 11 Testing

Summary of Chapter 12: Testing

Importance of Testing

Testing is vital to a project's lifecycle, offering confidence, documentation, and ease in refactoring. Go provides strong testing primitives, but common mistakes can make testing processes inefficient, brittle, or inaccurate.

Common Mistakes in Testing

Not Categorizing Tests

Tests should be categorized for improved execution efficiency. The testing pyramid suggests most tests should be unit tests, which are fast and deterministic.

1.

Build Tags

: Use build tags to categorize tests and run specific types

More Free Book



Scan to Download



Listen It

(e.g., unit or integration).

2.

Environment Variables

: Enable skipping tests based on environment variables, making the reason explicit if tests are ignored.

3.

Short Mode

: Distinguish short-running tests from long-running ones, allowing the skipping of longer tests when needed.

Not Enabling the -race Flag

The race detector tool helps identify data races in concurrent Go programs. Forgetting to enable the -race flag while testing can lead to undetected issues.

Not Using Test Execution Modes

Install Bookey App to Unlock Full Text and Audio

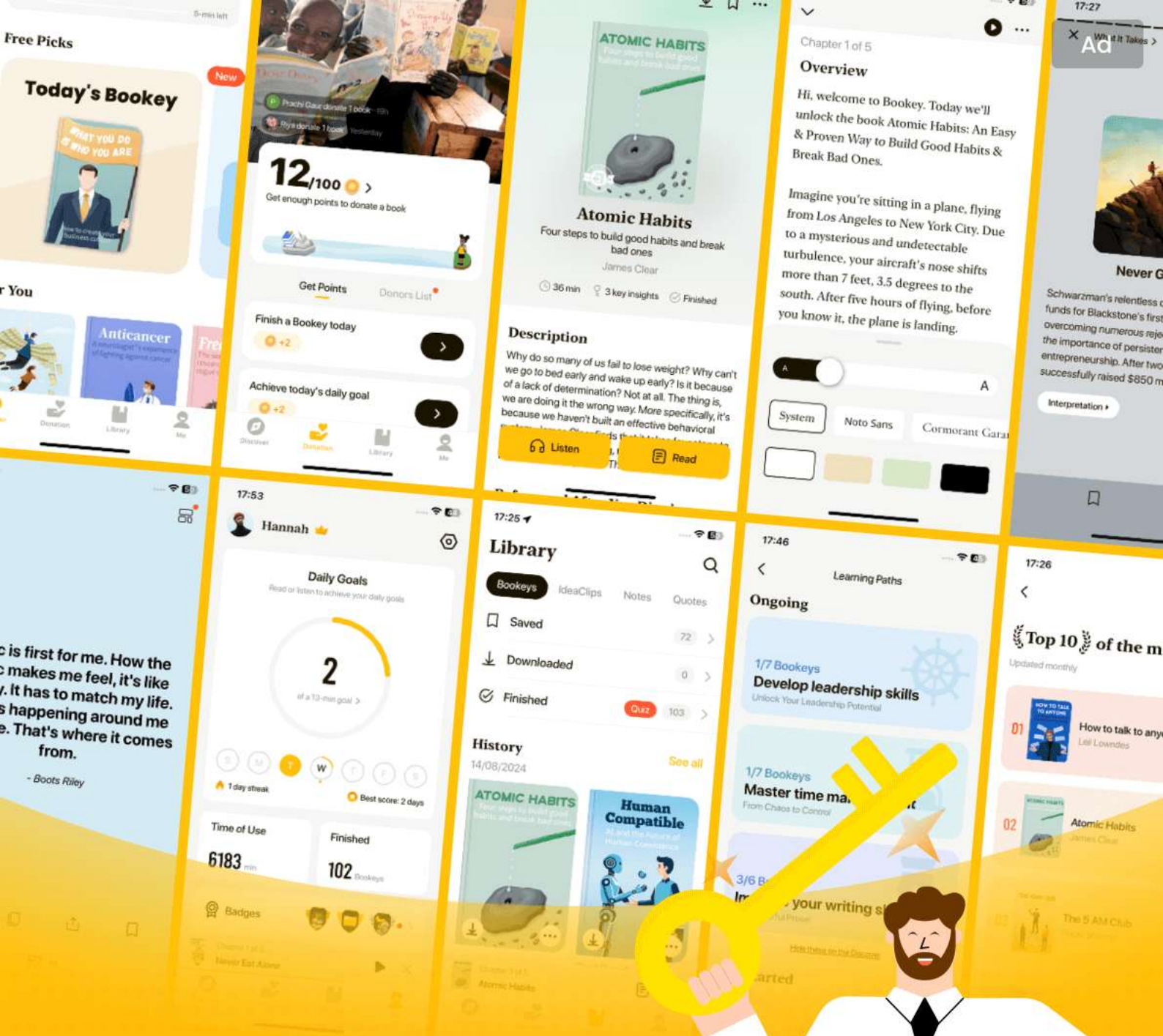
More Free Book



Scan to Download



Listen It



World's best ideas unlock your potential

Free Trial with Bookey



Scan to download



Chapter 13 Summary : 12 Optimizations

Chapter 13 Summary: Optimizations

In this chapter, we explore various optimization strategies in Go, emphasizing readability and clarity of code before performance. The focus is on understanding the underlying architecture, such as CPU caches, and employing diagnostic tools to identify bottlenecks.

12.1 Understanding CPU Caches

-

Mechanical Sympathy:

Familiarizing ourselves with how systems are designed can lead to better performance in Go applications. Understanding CPU caches (L1, L2, L3) and their architectures helps optimize memory usage.

-

Cache Lines and Locality:

Efficient memory access relies on cache lines. The principles of temporal and spatial locality dictate how data should be

More Free Book



Scan to Download



Listen It

accessed sequentially to make optimal use of caches.

-

Slice of Structs vs. Struct of Slices:

The organization of data impacts cache performance. Structs arranged by size lead to better spatial locality and faster processing.

-

Predictability in Memory Access:

Programming with predictable data access patterns enables the CPU to optimize its execution, reducing latency.

-

Cache Placement Policy:

Understanding cache strategies affects performance. For instance, certain memory access patterns can lead to cache misses.

12.2 Writing Concurrent Code

-

False Sharing:

Be aware of how concurrent modifications affect performance. Mismanagement of shared memory can slow down applications significantly.

More Free Book



Scan to Download



Listen It

12.3 Instruction-Level Parallelism (ILP)

- ILP allows CPUs to execute multiple instructions simultaneously. Code should be crafted to minimize data hazards that inhibit concurrent execution.

12.4 Data Alignment

- Proper data alignment is crucial for optimal memory access and preventing unnecessary memory consumption and performance degradation.

12.5 Stack vs. Heap Storage

-

Memory Allocation:

The stack is faster and self-cleaning, while heap space necessitates garbage collection (GC). Understanding when and why data is allocated to the heap can optimize application performance.

-

Escape Analysis:

Compiler decisions about stack vs. heap allocation impact application efficiency.

More Free Book



Scan to Download



Listen It

12.6 Reducing Allocations

- Strategies such as changing APIs, leveraging compiler optimizations, and using sync.Pool help reduce costly memory allocations.

12.7 Inlining Functions

- Inline functions reduce call overhead and can enable further optimizations by the compiler. Understanding inlining can help optimize application performance.

12.8 Go Diagnostics Tooling

- Profiling and tracing help developers visualize and optimize application performance through insights into CPU usage and memory allocation.

12.9 Understanding Garbage Collector Behavior

- Knowledge of the Go GC's mechanisms and tuning its parameters (e.g., GOGC) is essential, especially during application load peaks.

More Free Book



Scan to Download



Listen It

12.10 Running Go in Docker and Kubernetes

- Be aware of the implications of resource limits when deploying Go applications in containerized environments. Use libraries like `automaxprocs` to mitigate issues related to CPU throttling.

Final Words:

The key to effective optimizations lies in a deep understanding of system architecture, careful API design, and utilization of Go's diagnostic tools for profiling and performance monitoring. Learning from common mistakes can significantly enhance application performance and maintainability.

More Free Book



Scan to Download



Listen It

Ad



Scan to Download



Try Bookey App to read 1000+ summary of world best books

Unlock **1000+** Titles, **80+** Topics

New titles added every week

Brand



Leadership & Collaboration



Time Management



Relationship & Communication



Business Strategy



Creativity



Public



Money & Investing



Know Yourself



Positive Psychology

Entrepreneurship



World History



Parent-Child Communication

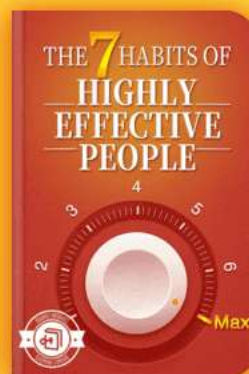
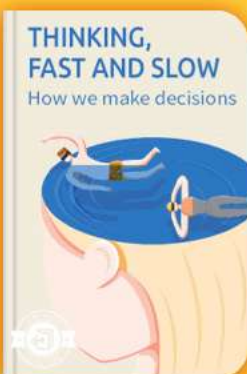


Self-care



Mind & Spirituality

Insights of world best books



Free Trial with Bookey



Best Quotes from 100 Go Mistakes and How to Avoid Them by Teiva Harsanyi with Page Numbers

[View on Bookey Website and Generate Beautiful Quote Images](#)

Chapter 1 | Quotes From Pages 3-41

1. I realized that communicating about common mistakes was a powerful tool.
2. It took me almost 2 years and countless iterations to frame each of the 100 mistakes alongside meaningful examples and multiple solutions where context is key.
3. I hope this book will help you avoid making these common mistakes and help you enhance your proficiency in the Go language.
4. This book is for developers with existing knowledge of the Go language.
5. Go: Simple to learn but hard to master.

Chapter 2 | Quotes From Pages 42-70

1. A person who never made a mistake never tried anything new.

More Free Book



Scan to Download



[Listen It](#)

2.Simple doesn't mean easy.

3.Tell me and I forget. Teach me and I remember. Involve me and I learn.

4.The ratio of time spent reading versus writing is well over 10 to 1.

Chapter 3 | Quotes From Pages 71-197

1.Organizing a Go codebase in a clean, idiomatic, and maintainable manner isn't an easy task. It requires experience and even mistakes to understand all the best practices related to code and project organization.

2.Variable shadowing occurs when a variable name is redeclared in an inner block, but we saw that this practice is prone to mistakes.

3.When an if block returns, we should omit the else block in all cases.

4.Interfaces are made to create abstractions. And the main caveat when programming meets abstractions is remembering that abstractions should be discovered, not

More Free Book



Scan to Download



Listen It

created.

- 5.If we find a need for getters and setters or, as mentioned, foresee a future need while guaranteeing forward compatibility, there's nothing wrong with using them.
- 6.Every exported element must be documented. Whether it is a structure, an interface, a function, or something else, if it's exported, it must be documented.
- 7.Using type embedding can also help avoid boilerplate code; however, ensure that doing so doesn't lead to visibility issues where some fields should have remained hidden.

More Free Book



Scan to Download



Listen It



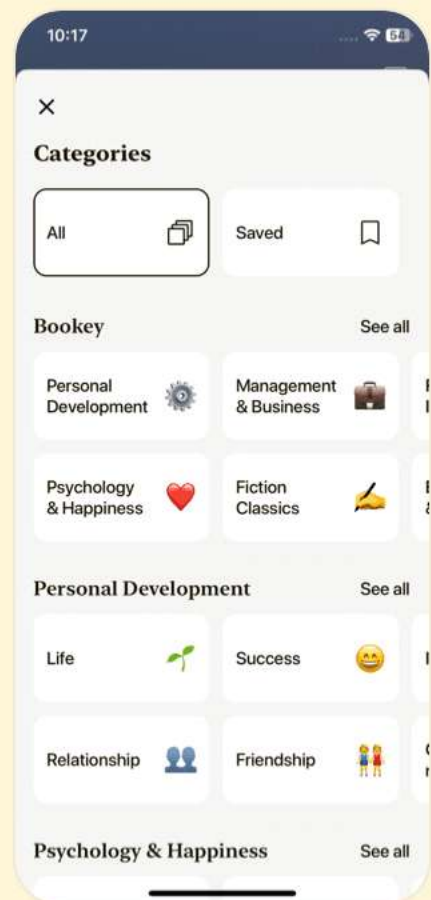
Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Scan to Download



Chapter 4 | Quotes From Pages 198-296

1. Dealing with data types is a frequent operation for software engineers.
2. Octal numbers start with a 0.
3. An integer overflow occurs when an arithmetic operation creates a value outside the range that can be represented with a given number of bytes.
4. Float32 and float64 are approximations of real arithmetic.
5. The slice length is the number of available elements in the slice, whereas the slice capacity is the number of elements in the backing array.
6. If we want to check for overflows, we can use the utility functions described in this section.
7. A nil slice is empty, whereas an empty slice isn't necessarily nil.
8. A map can always grow in memory, but it never shrinks.
9. Using reflect.DeepEqual has a specific behavior depending on the type we provide.
10. In most cases, using reflect.DeepEqual is a solution, but

More Free Book



Scan to Download



Listen It

be aware of the performance penalty.

Chapter 5 | Quotes From Pages 297-346

1. The value element in a range loop is a copy.

Therefore, to mutate a struct, for example, access it via its index or via a classic for loop (unless the element or the field you want to modify is a pointer).

2. Using break or continue with a label enforces breaking a specific statement. This can be helpful with switch or select statements inside loops.

3. When using defer, we must remember that it schedules a function call when the surrounding function returns. Hence, calling defer within a loop will stack all the calls: they won't be executed during each iteration, which may cause memory leaks if the loop doesn't terminate, for example.

4. The range operator creates a copy of the array. Meanwhile, the loop doesn't update the copy; it updates the original array: a. Therefore, the value of v during the last iteration is 2, not 10.

More Free Book



Scan to Download



Listen It

5.It's essential to keep these behaviors in mind so we don't base our code on wrong assumptions.

Chapter 6 | Quotes From Pages 347-379

1. Understanding that a rune corresponds to the concept of a Unicode code point and that it can be composed of multiple bytes should be part of the Go developer's core knowledge to work accurately with strings.
- 2.To access a specific rune index (such as the third rune), convert the string into a `[]rune`.
- 3.Concatenating a list of strings should be done with `strings.Builder` to prevent allocating a new string during each iteration.
- 4.Using copies instead of substrings can prevent memory leaks, as the string returned by a substring operation will be backed by the same byte array.
- 5.The `bytes` package offers the same operations as the `strings` package can help avoid extra byte/string conversions.

More Free Book



Scan to Download



Listen It



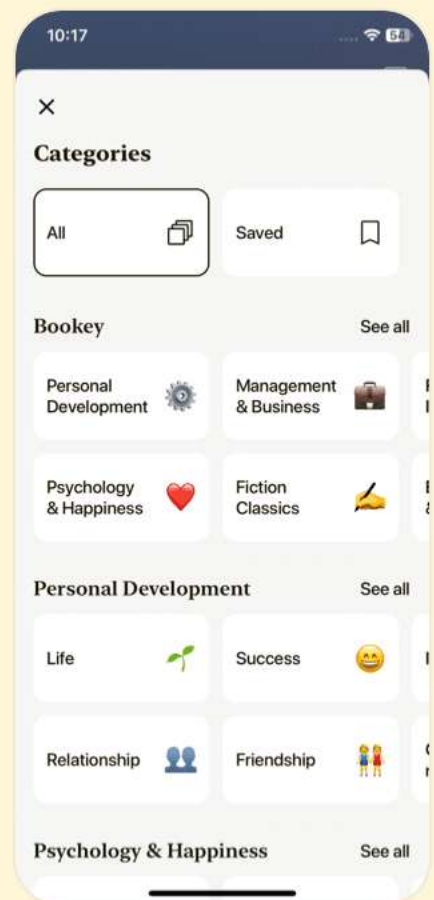
Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Scan to Download



Chapter 7 | Quotes From Pages 380-421

1. In Go, we can attach either a value or a pointer receiver to a method.
2. Choosing between value and pointer receivers isn't always straightforward.
3. Named result parameters can improve code readability in some cases and be quite handy in others.
4. Using named result parameters doesn't necessarily mean using naked returns.
5. In Go, a nil pointer is a valid receiver.
6. Accepting a filename as a function input should, in most cases, be considered a code smell.
7. When we call defer on a function or method, the call's arguments are evaluated immediately.

Chapter 8 | Quotes From Pages 422-464

1. In Go, panic is a built-in function that stops the ordinary flow.
2. Panicking in Go should be used sparingly.
3. error management is a fundamental aspect of building



robust and observable applications, and it should be as important as any other part of a codebase.

- 4.If we want to prevent it, we shouldn't use error wrapping but error transformation, for example, using `fmt.Errorf` with the `%v` directive.
- 5.Handling an error should be done only once.
- 6.Ignoring an error in Go should be the exception.
- 7.Using error wrapping is the most convenient approach as it allows us to propagate the source error and add context to an error.

Chapter 9 | Quotes From Pages 465-539

- 1.Concurrency is about structure, and we can change a sequential implementation into a concurrent one by introducing different steps that separate concurrent threads can tackle.

Meanwhile, parallelism is about execution, and we can use it at the step level by adding more parallel threads.

- 2.A misconception among many developers is believing that

More Free Book



Scan to Download



Listen It

a concurrent solution is always faster than a sequential one.

This couldn't be more wrong.

3. Knowing the workload type (I/O or CPU) is crucial when designing concurrent applications.

4. Understanding the fundamental differences between concurrency and parallelism is a cornerstone of the Go developer's knowledge.

5. Concurrency isn't always faster and shouldn't be considered the default way to go for all problems.

6. A data race occurs when two or more goroutines simultaneously access the same memory location and at least one is writing.

7. Understanding both data races and race conditions is crucial to becoming proficient in designing concurrent applications.

More Free Book



Scan to Download



Listen It



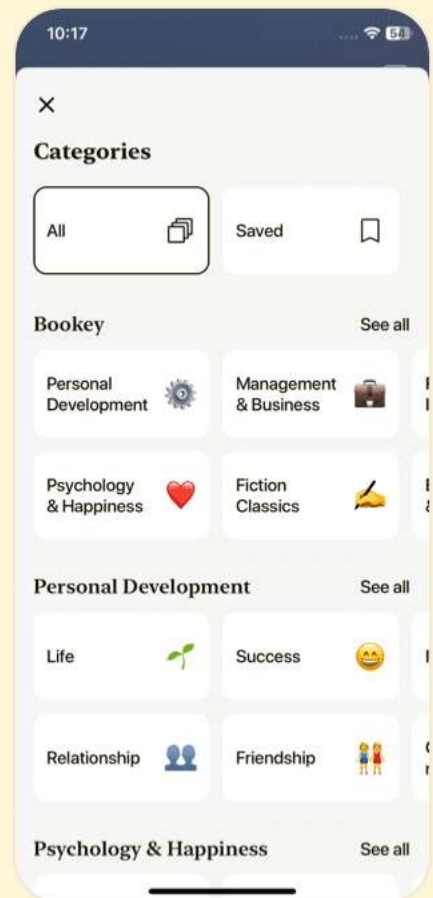
Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Scan to Download



Chapter 10 | Quotes From Pages 540-634

1. Propagating a context should be done cautiously.
2. Starting a goroutine without knowing when to stop it is a design issue.
3. Mishandling goroutines and loop variables is probably one of the most common mistakes made by Go developers.
4. When using select with multiple channels, we must remember that if multiple options are possible, the first case in the source order does not automatically win.
5. In Go, an empty struct is a de facto standard to convey an absence of meaning.
6. Nil channels are useful in some conditions and should be part of the Go developer's toolset when dealing with concurrent code.
7. Deciding about an accurate queue size isn't an easy problem.
8. We should remain cautious about the possible side effects of string formatting.
9. Whenever multiple goroutines have to access a common



sync element, we must ensure that they all rely on the same instance.

10. When using a `sync.WaitGroup`, the `Add` operation must be done before spinning up a goroutine in the parent goroutine.

Chapter 11 | Quotes From Pages 635-713

1. The Go standard library is a set of core packages that enhance and extend the language.
2. When using `sql.Open`, one common misconception is expecting this function to establish connections to a database.
3. For production-grade applications, we must consider these four parameters: `SetMaxOpenConns`, `SetMaxIdleConns`, `SetConnMaxIdleTime`, and `SetConnMaxLifetime`.
4. We should be careful with embedded fields. While promoting the fields and methods of an embedded field type can sometimes be convenient, it can also lead to subtle bugs because it can make the parent struct implement interfaces without a clear signal.



5. We always need to remember that `http.Error` doesn't stop a handler execution and must be added manually.
6. For production-grade systems, we probably want to override the default timeouts.

Chapter 12 | Quotes From Pages 714-816

1. Testing is a crucial aspect of a project's lifecycle.
2. Throughout this chapter, we look at common mistakes that make the testing process brittle, less effective, and less accurate.
3. Most tests should be unit tests: they're cheap to write, fast to execute, and highly deterministic.
4. Using the `-race` flag is highly recommended when writing concurrent applications.
5. Table-driven tests are an efficient technique for writing condensed tests and thus reducing boilerplate code to help us focus on what matters: the testing logic.
6. A flaky test is a test that may both pass and fail without any code change.
7. In general, we should be cautious about micro-benchmarks.



8. Not exploring all the Go testing features leads to a less accurate and efficient testing process.

More Free Book



Scan to Download



Listen It



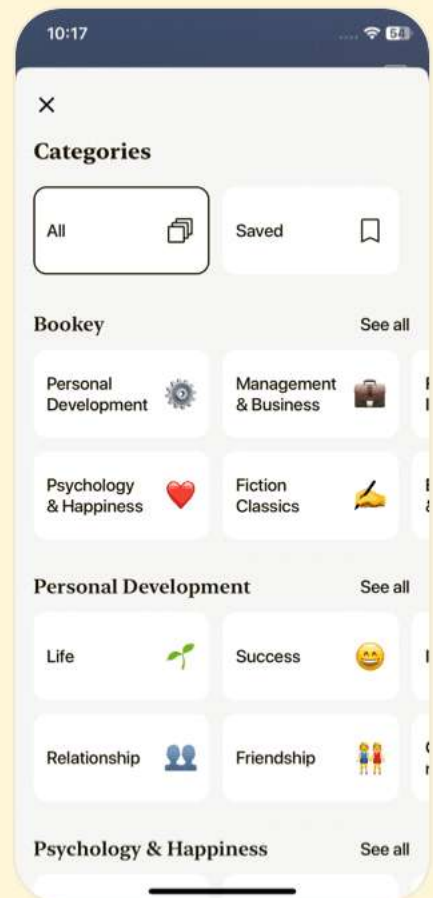
Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Scan to Download



Chapter 13 | Quotes From Pages 817-944

1. Make it correct, make it clear, make it concise, make it fast, in that order.
2. Mechanical sympathy is a term coined by Jackie Stewart, a three-time F1 world champion: You don't have to be an engineer to be a racing driver, but you do have to have mechanical sympathy.
3. The closer a memory location is to a logical core, the faster accesses are.
4. Why does this matter? Because it allows the CPU to increase the level of parallelism.
5. Understanding how to use CPU caches is important for optimizing CPU-bound applications because the L1 cache is about 50 to 100 times faster than the main memory.
6. Stack allocations are almost free, whereas heap allocations are slower and rely on the GC to clean the memory.
7. We should generally favor unit strides over constant strides because of the better spatial locality.

More Free Book



Scan to Download



Listen It



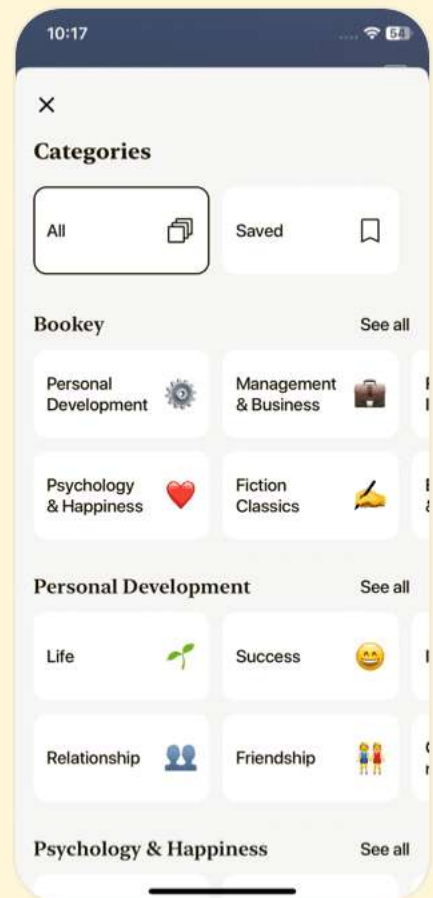
Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Scan to Download



100 Go Mistakes and How to Avoid Them Questions

[View on Bookey Website](#)

Chapter 1 | 100 Go Mistakes| Q&A

1.Question

What inspired Teiva Harsanyi to write this book on Go mistakes?

Answer:Teiva Harsanyi was inspired to write this book after observing common coding mistakes during his second professional experience with Go in 2019. His popular blog post, 'The Top 10 Most Common Mistakes I've Seen in Go Projects,' highlighted these issues and received positive feedback from the Go community, leading him to believe that documenting these mistakes could be beneficial for developers.

2.Question

How did Teiva Harsanyi gather the 100 mistakes included in the book?

More Free Book



Scan to Download



[Listen It](#)

Answer:He compiled the 100 mistakes over about a year, researching various sources including professional projects, open source repositories, books, blogs, and discussions with the Go community. Additionally, he reflected on his own experiences as a source of inspiration for the mistakes.

3.Question

What is the target audience for this book?

Answer:The book is aimed at developers who already have some knowledge of the Go language, ideally those who have worked on existing Go projects, as it does not cover basic concepts like syntax or keywords.

4.Question

What kind of mistakes does Chapter 1 address?

Answer:Chapter 1, titled 'Go: Simple to learn but hard to master,' discusses why Go, despite its reputation for simplicity, poses challenges in mastering its intricacies. It sets the stage for the types of mistakes that will be addressed in the subsequent chapters.

5.Question

Can you explain how the book is organized?

More Free Book



Scan to Download



Listen It

Answer: The book is structured into 12 chapters, each focusing on different aspects of Go programming, such as project organization, data types, control structures, and concurrency. Each chapter highlights common mistakes and provides meaningful examples and solutions.

6.Question

Why is understanding common mistakes in Go programming important?

Answer: Understanding common mistakes in Go is crucial because it helps developers enhance their proficiency in the language, avoid pitfalls that could lead to ineffective or buggy code, and encourages best practices that lead to cleaner, more maintainable code.

7.Question

What role did feedback from the Go community play in the development of this book?

Answer: Feedback from the Go community was invaluable, as it not only validated Teiva Harsanyi's observations but also helped him refine the mistakes and solutions he

More Free Book



Scan to Download



Listen It

documented, ultimately leading to a more effective and insightful book.

8.Question

What are some key themes that the author aims to convey through this book?

Answer:Key themes include the importance of learning from mistakes, the complexity hidden behind Go's simplicity, and the value of collaboration and community input in professional growth and knowledge sharing among developers.

9.Question

How does the author define the difference between concurrency and parallelism in the context of Go?

Answer:While the book dives into concurrency in detail, it sets the performance groundwork by explaining that concurrency involves structuring programs to achieve tasks in overlapping time periods, whereas parallelism refers to executing tasks simultaneously, often utilizing multiple processors.

More Free Book



Scan to Download



Listen It

Chapter 2 | 1 Go: Simple to learn but hard to master| Q&A

1.Question

What does it mean that Go is 'simple to learn but hard to master'?

Answer:This phrase highlights that while Go has a straightforward syntax and can be picked up quickly by newcomers, mastering its deeper features—such as concurrency and best practices—requires significant time, experience, and effort. Just like learning to ride a bike is easy, but mastering advanced tricks takes practice.

2.Question

Why are mistakes considered valuable in the journey of learning Go?

Answer:Mistakes are crucial learning opportunities; they foster growth and understanding. As science shows, our brains develop best when we deal with errors, helping us remember not just the mistake but also the associated context. This book emphasizes learning from 100 common

More Free Book



Scan to Download



Listen It

Go mistakes to accelerate proficiency.

3.Question

What common mistakes do developers make in Go that affect code readability?

Answer:Common issues include using deeply nested code, failing to use named result parameters, and not considering how future readers will interpret the code. These mistakes can obscure the intent of the code, making it harder to maintain over time.

4.Question

Why is it important to avoid needless complexity in coding?

Answer:Developers often attempt to anticipate future needs, leading to over-engineered solutions that complicate the codebase. This complexity can hinder comprehension and maintenance, making it harder to solve current problems effectively.

5.Question

How does the design of Go relate to its stability and performance?

More Free Book



Scan to Download



Listen It

Answer:Go was designed with stability and quick compilation in mind, allowing developers to write code that is both reliable and efficient. Its limited feature set is a conscious choice to maintain clarity and speed, avoiding unnecessary complications.

6.Question

What role does concurrency play in Go, and why is it a complicated topic despite being simply learned?

Answer:Concurrency in Go, while presented through simple constructs like goroutines and channels, can lead to significant bugs if not understood fully. Developers must grasp the implications of different concurrency models and be aware of potential common traps to effectively utilize these features.

7.Question

What are the seven categories of mistakes that the book discusses?

Answer:The seven categories are: 1. Bugs 2. Needless complexity 3. Weaker readability 4. Suboptimal or

More Free Book



Scan to Download



Listen It

unidiomatic organization 5. Lack of API convenience 6.
Under-optimized code 7. Lack of productivity

8.Question

How can learning from errors improve programming skills?

Answer:Confronting mistakes encourages reflection and deeper understanding, enabling developers to approach future challenges more effectively. The book pairs each mistake with real-world examples, making the lessons more relatable and practical.

9.Question

What does the quote 'Tell me and I forget. Teach me and I remember. Involve me and I learn.' imply for programmers?

Answer:This quote emphasizes that true learning happens through active involvement and experience rather than passive absorption of information. For programmers, engaging with real-world problems and learning from mistakes leads to deeper comprehension and skill.

10.Question

More Free Book



Scan to Download



Listen It

What should developers focus on to maximize their productivity in Go?

Answer: Developers should focus on utilizing Go's standard library effectively, understanding its tooling for testing and profiling, and optimizing their coding practices to harness Go's strengths for quick and clean development.

Chapter 3 | 2 Code and project organization| Q&A

1.Question

What is variable shadowing and why should it be avoided?

Answer: Variable shadowing occurs when a variable declared in an inner block hides a variable with the same name declared in an outer block. This can lead to unintended consequences, such as always using a nil value from the outer variable when the inner variable is expected to be used. Avoiding shadowing prevents bugs and confusion, ensuring that the correct variable value is accessed.

2.Question

More Free Book



Scan to Download



Listen It

How can we enhance the readability of nested code?

Answer: To improve readability, we should reduce the number of nested levels. This can be achieved by avoiding unnecessary else blocks when an if statement returns early. Align the happy path of execution on the left for clearer scanning of code flow. Thus, restructuring code to eliminate excessive nesting keeps it simple and easier to understand.

3.Question

When should init functions be used and what are their risks?

Answer: Init functions can be used for initial setup of package state. However, they should not manage important app states like database connections due to limited error handling, making it hard to implement retries or fallback mechanisms. They may complicate testing as they execute before tests, potentially creating unwanted dependencies.

4.Question

What is the recommendation regarding the use of getters and setters in Go?

More Free Book



Scan to Download



Listen It

Answer: Getters and setters should not be overused in Go, as direct access to struct fields is encouraged. They should be used only when encapsulation is necessary or when future flexibility is needed. The focus should be on being pragmatic and not adhering to conventions without necessity.

5.Question

Why is interface pollution a problem in Go?

Answer: Interface pollution happens when too many unnecessary abstractions are created, making the codebase harder to navigate and understand. It is best to design interfaces based on actual needs rather than anticipated needs, thereby avoiding confusion and complexity in the code.

6.Question

What is an effective approach for dealing with optional configurations?

Answer: The functional options pattern is highly effective for managing optional configurations. This pattern allows builders to define and apply configuration options flexibly,

More Free Book



Scan to Download



Listen It

enhancing usability and readability in function calls without breaking existing functionality.

7.Question

How should Go projects be structured for better maintainability?

Answer: Adopting a project structure like the project-layout promotes robust organization within Go projects. This layout provides clear separation of components, making maintenance easier. Consistency in organization also aids in understanding the codebase across different teams.

8.Question

What is considered a better alternative to utility packages like 'utils' or 'common'?

Answer: Creating descriptive, specific package names that reflect their function rather than generic names like 'utils' is essential. For example, instead of a utilities package, a dedicated 'stringset' package would clearly convey its purpose and functionality.

9.Question

How can documentation improve the usability of Go

More Free Book



Scan to Download



Listen It

code?

Answer: Documentation enhances code usability by explaining the purpose and usage of exported elements. It should be concise, well-structured, and clear enough that developers can understand the functionality without needing to read through the source code.

10.Question

What are the best practices regarding variable and package name collisions in Go?

Answer: To avoid confusion, package names should not collide with variable names. If a collision occurs, the variable name should be changed or an alias for the package import should be used to differentiate between them. This prevents ambiguity in code reading and maintenance.

11.Question

Why should linters be utilized in Go development?

Answer: Linters are essential tools for analyzing code to catch potential errors, enforce style guidelines, and improve code quality. They can detect issues such as variable shadowing

More Free Book



Scan to Download



Listen It

and ensure that code adheres to best practices without manual checking.

More Free Book



Scan to Download



Listen It



Scan to Download



Why Bookey is must have App for Book Lovers



30min Content

The deeper and clearer interpretation we provide, the better grasp of each title you have.



Text and Audio format

Absorb knowledge even in fragmented time.



Quiz

Check whether you have mastered what you just learned.



And more

Multiple Voices & fonts, Mind Map, Quotes, IdeaClips...

Free Trial with Bookey



Chapter 4 | 3 Data types| Q&A

1.Question

What is the significance of octal literals in Go, and how should they be used to prevent confusion?

Answer:Octal literals in Go start with a '0' and are interpreted as base 8 numbers. To prevent confusion, especially when reading code, it's recommended to explicitly use the '0o' prefix to indicate an octal value (e.g., 0o644 for file permissions). This clarity helps avoid bugs related to misinterpretation of numeric bases.

2.Question

Why should developers be cautious about integer overflows during arithmetic operations?

Answer:Integer overflows can lead to silent bugs; for instance, when adding integers, exceeding the maximum value can flip the sign of the result unexpectedly (e.g., from positive to negative). Developers should implement checks, like testing against `math.MaxInt`, to prevent unintended



consequences.

3.Question

How do floating-point inaccuracies affect programming, and what strategies can mitigate these issues?

Answer: Floating-point values are approximations and can suffer from precision loss, which especially impacts comparisons and accumulations. To mitigate this, rather than using '==' for comparison, developers should check if the difference between two values is smaller than a defined tolerance (delta). Additionally, organizing calculations to group similar magnitudes first can enhance accuracy.

4.Question

What are the key differences between slice length and capacity in Go, and why is this distinction important?

Answer: Length is the number of elements currently in the slice, while capacity is the total number of elements the slice can accommodate without reallocating memory.

Understanding this distinction aids in optimizing memory usage and preventing memory leaks when appending items;

More Free Book



Scan to Download



Listen It

it also leads to better performance by minimizing allocations.

5.Question

How can the initialization of a slice with a known length or capacity enhance performance?

Answer:When initializing a slice with a known length or capacity, Go allocates the appropriate amount of memory upfront, which avoids multiple reallocations and copying of elements as items are added. This can significantly increase performance, especially in loops or batch processing scenarios.

6.Question

What is the difference between nil and empty slices in Go, and how does this impact memory allocation?

Answer:A nil slice is a slice that hasn't been initialized and points to no memory, requiring no allocations, while an empty slice has been allocated but contains no elements (its length is zero). It's important to return nil slices when appropriate to save on memory allocation costs and to prevent unintended behavior in serialization and

More Free Book



Scan to Download



Listen It

comparisons.

7.Question

What should developers remember when copying slices in Go?

Answer:When using the copy built-in function, the number of copied elements will always be the minimum of the lengths of the source and destination slices. To actually copy all elements from a source slice, the destination slice must have been initialized to the appropriate length.

8.Question

What precautions should be taken when using the append function with slices to avoid unexpected side effects?

Answer:To prevent unintended mutations to original slices, especially when slicing operations are involved, it's advisable to either create a copy of the slice or use a full slice expression (e.g., `s[:2:2]` to limit the capacity). This ensures that appending operations do not overwrite other values unexpectedly.

9.Question

What are the implications of a Go map's inability to

More Free Book



Scan to Download



Listen It

shrink in size when it comes to memory management?

Answer:A Go map's inability to shrink means that even after deleting elements, the memory allocated for the map may not decrease, possibly leading to unnecessary memory bloat.

Developers should consider periodically re-creating maps or using pointer types for values to optimize memory consumption under high load.

10.Question

How should value comparison be conducted in Go when using non-comparable types?

Answer:For comparing non-comparable types, developers can use the `reflect.DeepEqual` function to check for deep equality, although it comes with performance overhead.

Alternatively, implementing custom comparison methods for specific types can provide a more efficient and tailored approach.

Chapter 5 | 4 Control structures| Q&A

1.Question

What should we remember about the value element in a

More Free Book



Scan to Download



Listen It

range loop in Go?

Answer: The value element is a copy. If we want to mutate a struct within a range loop, we need to access it via its index or use a classic for loop, unless the value being modified is a pointer.

2.Question

How is the expression passed to a range loop evaluated?

Answer: The expression is evaluated only once before the loop begins, resulting in a copy used during the iterations. This is crucial to avoid common mistakes, such as trying to modify the original slice during the iteration.

3.Question

What happens if we try to append to a slice while using a range loop on it?

Answer: The loop will complete after iterating over the elements initially present in the slice, ignoring any additions made to the slice during the iteration. This is because the slice is evaluated once at the start.

4.Question

What is a common mistake developers make when using

More Free Book



Scan to Download



Listen It

pointers in range loops?

Answer:A common mistake is storing a pointer to a variable from the loop, which results in all stored pointers pointing to the same instance rather than unique instances, due to the reuse of the same variable address.

5.Question

Why shouldn't we rely on the ordering of elements in a map while iterating?

Answer:Maps in Go do not maintain order; iteration can yield elements in random order, differing each time we iterate, which means we cannot rely on any specific order of keys when iterating.

6.Question

What happens if we update a map while iterating over it?

Answer:While it is allowed to update a map during iteration, it can lead to unpredictable results, where new entries may or may not appear in the current iteration.

7.Question

How can we properly break out of a for loop that contains a switch statement?

More Free Book



Scan to Download



Listen It

Answer: We should use a labeled break statement that specifically targets the for loop, ensuring that we exit the correct loop rather than just the innermost switch.

8.Question

What is a significant risk of using defer inside a loop?

Answer: Using defer inside a loop can lead to resource leaks because defers are executed only when the surrounding function returns, which may never happen if the loop doesn't terminate.

9.Question

How can we handle file closures properly when using defer in a loop?

Answer: To avoid keeping file descriptors open, we can either extract the file handling logic into a separate function or use a closure to ensure that defer calls happen at each iteration.

10.Question

What are the best practices to avoid the common mistakes discussed in this chapter?

Answer: Developers should remember that the ranged value in loops is a copy, understand how expressions are evaluated

More Free Book



Scan to Download



Listen It

before iteration, avoid relying on map ordering, and manage resources correctly when using defer.

Chapter 6 | 5 Strings| Q&A

1.Question

What is a rune in Go and why is it important for understanding strings?

Answer:A rune in Go is defined as a Unicode code point, effectively an alias for int32. It is crucial for understanding how strings are handled since Go strings are composed of sequences of bytes, with runes potentially made up of multiple bytes.

Understanding runes helps programmers manage string operations accurately, preventing iterations or manipulations that could lead to unexpected results when handling multi-byte characters.

2.Question

How does Go handle the length of strings in terms of bytes versus runes?

Answer:In Go, the built-in 'len' function returns the number

More Free Book



Scan to Download



Listen It

of bytes in a string rather than the number of runes. For example, if you have a string containing characters that are encoded with multiple bytes, like `ℓ` (which 'len' will report 3 instead of 1. This distinction is essential for correctly iterating and manipulating strings containing Unicode characters.

3.Question

What are some common mistakes developers make when iterating over strings?

Answer:A common mistake is assuming that iterating with `for i := range s` will provide direct access to each rune. Instead, it provides the starting index of each rune's byte sequence. To correctly access and print each rune, developers should either use the range's 'value' variable or convert the entire string to a slice of runes.

4.Question

What is the difference between TrimRight and TrimSuffix functions in Go?

Answer:TrimRight removes all specified trailing runes in a

More Free Book



Scan to Download



Listen It

set until encountering a rune not in the set, while TrimSuffix removes a specific suffix from the end of a string. For example, TrimRight('123oxo', 'xo') will return '123', while TrimSuffix('123oxo', 'xo') will return '123o'. Understanding this difference is crucial to avoid unintended results.

5.Question

Why is using strings.Builder more efficient for concatenating strings?

Answer:Using strings.Builder avoids the inefficiency of reallocation. The '+' operator in string concatenation creates a new string each time, due to strings being immutable. Strings.Builder minimizes memory copying by maintaining an internal buffer that grows efficiently, allowing it to allocate memory once instead of multiple times during concatenation.

6.Question

How can unnecessary string conversions lead to performance issues?

Answer:Unnecessary conversions between []byte and string

More Free Book



Scan to Download



Listen It

can significantly impact performance. Each conversion involves memory allocation and copying of bytes, which can add overhead, especially in I/O operations. Instead, working directly with []byte where possible can reduce these conversions and improve efficiency.

7.Question

What potential memory leak issue arises from using substrings in Go?

Answer:Using substrings can lead to memory leaks because a substring shares the same backing array as the original string. If the original string is large, storing a substring reference could inadvertently keep the entire original string in memory. To avoid this, developers should make a copy of the substring using []byte conversion.

8.Question

How can developers avoid mixing up string manipulation functions in the strings package?

Answer:To avoid confusion, it's important to clearly understand the purpose of each function. Developers should

More Free Book



Scan to Download



Listen It

keep the intended behavior of TrimLeft, TrimRight, TrimPrefix, and TrimSuffix in mind and verify the expected results through documentation or examples. Regular practice and familiarity with these functions can also help prevent mix-ups.

9.Question

What is the recommended approach for accessing individual runes in a string?

Answer:To access individual runes, developers should convert the string into a slice of runes (using []rune(s)) and then use indexing. This allows for accurate access to the ith rune, maintaining awareness that each rune might consist of multiple bytes.

10.Question

What should developers consider when optimizing string handling in Go?

Answer:Developers should consider the immutability of strings, the performance of different concatenation methods, the potential efficiency of using the bytes package, and the

More Free Book



Scan to Download



Listen It

implications of substring operations regarding memory usage. Awareness of these factors can greatly enhance performance while minimizing common mistakes.

More Free Book



Scan to Download



Listen It

Ad



Scan to Download



App Store
Editors' Choice



22k 5 star review

Positive feedback

Sara Scholz

...tes after each book summary
...erstanding but also make the
...and engaging. Bookey has
...ding for me.

Fantastic!!!



I'm amazed by the variety of books and languages
Bookey supports. It's not just an app, it's a gateway
to global knowledge. Plus, earning points for charity
is a big plus!

Masood El Toure

Fi



Ab
bo
to
my

José Botín

...ding habit
...o's design
...ual growth

Love it!



Bookey offers me time to go through the
important parts of a book. It also gives me enough
idea whether or not I should purchase the whole
book version or not! It is easy to use!

Wonnie Tappkx

Time saver!



Bookey is my go-to app for
summaries are concise, ins
curated. It's like having acc
right at my fingertips!

Awesome app!



I love audiobooks but don't always have time to listen
to the entire book! bookey allows me to get a summary
of the highlights of the book I'm interested in!!! What a
great concept !!!highly recommended!

Rahul Malviya

Beautiful App



This app is a lifesaver for book lovers with
busy schedules. The summaries are spot
on, and the mind maps help reinforce wh
I've learned. Highly recommend!

Alex Walk

Free Trial with Bookey



Chapter 7 | 6 Functions and methods| Q&A

1.Question

What is the difference between value receivers and pointer receivers in Go?

Answer: Value receivers create a copy of the value when the method is called, meaning changes made within the method do not affect the original instance.

Pointer receivers, on the other hand, pass the address of the instance to the method, allowing modifications made within the method to reflect on the original instance.

2.Question

When should you use a pointer receiver?

Answer: You should use a pointer receiver when your method needs to mutate the receiver, if the receiver is a large object to avoid copying, or if it is a type that cannot be copied (like those containing sync types).

3.Question

What are named result parameters and when should they be used?

More Free Book



Scan to Download



Listen It

Answer:Named result parameters allow you to attach names to return values in a function, making them behave like regular variables. They should be used when they enhance readability, especially when the returned values need to be explicitly understood, such as in defining geographic coordinates (like latitude and longitude).

4.Question

What is a common pitfall when using named result parameters?

Answer:A common pitfall is that named result parameters are initialized to their zero values. This means they can lead to unintended side effects if you assume they hold meaningful data without checking their state before return.

5.Question

Why is returning a nil receiver problematic in Go?

Answer:Returning a nil receiver can lead to returning a non-nil interface value, which may cause confusion. The caller might expect a nil value but yields a non-nil because Go treats a nil pointer as its underlying type when

More Free Book



Scan to Download



Listen It

implementing interfaces.

6.Question

What is the recommended practice instead of passing a filename to a function?

Answer:Instead of passing a filename, it's better to accept an `io.Reader` as a function input. This practice improves code reusability as it allows the same function to handle various data sources such as files, HTTP requests, etc.

7.Question

What should you know about the evaluation of defer arguments in Go?

Answer:In Go, when you use `defer`, the arguments for the deferred function are evaluated immediately, not at the time the surrounding function returns. This means that if those arguments are variables that change later in the function, they won't reflect the updated values during the deferred call.

8.Question

What techniques can help manage defer arguments effectively?

Answer:You can either pass a pointer to the variable to be

More Free Book



Scan to Download



Listen It

modified or use a closure wrapping the deferred function. This ensures that the most recent value is referenced when the deferred calls are executed.

9.Question

How does mixing receiver types in methods affect code?

Answer:Mixing receiver types (value and pointer) in a struct's methods can create confusion and should generally be avoided. A consistent choice enhances clarity and intention behind the method's behavior.

10.Question

What should developers remember about the default choice of receivers in Go?

Answer:As a default rule, prefer value receivers unless you have a compelling reason to choose pointer receivers, such as mutability needs or performance considerations.

Chapter 8 | 7 Error management| Q&A

1.Question

What is a common mistake that developers make regarding panic in Go?

Answer:Many developers new to Go mistakenly use

More Free Book



Scan to Download



Listen It

panic to handle errors, replicating try/catch from other languages. Panic should only be used in truly exceptional situations like signaling programmer errors or when mandatory dependencies fail to initialize.

2.Question

When should errors be wrapped in Go?

Answer:Errors should be wrapped to add context or to mark them as specific types. For instance, when adding user context to a 'permission denied' error or when marking an error associated with HTTP 'Forbidden' responses.

3.Question

What should you do before checking for a specific error type after wrapping an error using %w?

Answer:You should use the 'errors.As' function to check whether the wrapped error is of a specific type, to avoid issues where the type check fails due to the wrapping.

4.Question

Why is ignoring an error without acknowledging it problematic?

More Free Book



Scan to Download



Listen It

Answer: Ignoring an error without explicit acknowledgment can confuse future developers about whether the omission was intentional. Always assign ignored errors to the blank identifier to show intent.

5.Question

How should errors from defer statements be handled?

Answer: Errors from defer statements should not be ignored. They can be logged, or if necessary, propagated to the caller using named result parameters to ensure proper error management.

6.Question

What is a better practice than handling an error multiple times?

Answer: An error should be handled only once; decide whether to log or return the error but avoid both. This makes debugging easier and keeps the code clean.

7.Question

What are sentinel errors, and when should they be used?

Answer: Sentinel errors are global error variables that represent expected errors. They should be used for

More Free Book



Scan to Download



Listen It

anticipated error conditions while unexpected errors should use specific error types.

8.Question

How can error wrapping affect error comparison?

Answer:Error wrapping can complicate direct comparisons (using `==`) of errors. It's important to use `'errors.Is'` for comparing wrapped errors to check if the error is of a specific sentinel type.

9.Question

What is the appropriate way to indicate an error should be ignored in Go?

Answer:When you choose to ignore an error, you should explicitly assign it to the blank identifier (e.g., `'_ = notify()'`) to convey intent to future readers.

10.Question

What is the primary conclusion about error handling in Go?

Answer:Error handling in Go emphasizes robustness, with clear strategies for panicking, wrapping, ignoring, and checking errors. Each case should be handled with care to

More Free Book



Scan to Download



Listen It

ensure clarity and proper function of the code.

Chapter 9 | 8 Concurrency: Foundations| Q&A

1.Question

What is the difference between concurrency and parallelism?

Answer:Concurrency is about structuring a system into independent threads that can work on different parts of a task simultaneously, while parallelism refers to executing multiple tasks at the same time to improve performance.

2.Question

Why do developers mistakenly think concurrency is always faster?

Answer:Many developers mistakenly believe that concurrent code will always outperform sequential code, but the actual performance depends on the nature of the workload, the implementation efficiency, and overhead costs like context switching.

3.Question

How does Go scheduling work with goroutines?

More Free Book



Scan to Download



Listen It

Answer:Go scheduling involves the runtime managing goroutines, where goroutines are assigned to OS threads, and the system aims to utilize CPU cores efficiently. Goroutines have a simpler lifecycle than OS threads and can be scheduled based on their states (executing, runnable, or waiting).

4.Question

How can a parallel version of a program sometimes be slower than its sequential counterpart?

Answer:If the tasks being parallelized are too small, the overhead of creating goroutines and context switching can surpass the performance gains achieved from executing tasks in parallel, making the sequential version faster.

5.Question

What determines the optimal number of goroutines for CPU-bound vs I/O-bound tasks?

Answer:The optimal number of goroutines for CPU-bound tasks is generally close to GOMAXPROCS (the number of available CPU cores), whereas for I/O-bound tasks, it's

More Free Book



Scan to Download



Listen It

influenced by the external system's ability to handle concurrent requests.

6.Question

When should you prefer channels over mutexes in Go?

Answer:Channels should be used for communication and orchestration between concurrent goroutines, while mutexes are more appropriate for synchronizing access to shared state among parallel goroutines.

7.Question

What are data races and race conditions?

Answer:A data race occurs when two or more goroutines simultaneously access the same memory location, and at least one is writing, leading to unpredictable results. A race condition, on the other hand, is a situation where the timing of events affects the outcome, even if there are no data races.

8.Question

Why is it important to understand the Go memory model?

Answer:The Go memory model defines the conditions under which reads and writes to variables are guaranteed to be

More Free Book



Scan to Download



Listen It

synchronized across multiple goroutines, helping developers avoid data races and ensure predictable behavior.

9.Question

What role do contexts play in Go concurrent programs?

Answer:Contexts in Go carry deadlines, cancellation signals, and key-value pairs across API boundaries, allowing for better management of resource utilization and coordination of goroutines.

10.Question

How does understanding workload type impact concurrency design?

Answer:Recognizing whether a workload is CPU-bound or I/O-bound influences how concurrency is implemented, specifically in terms of efficiency in managing goroutine counts and ensuring optimal performance.

More Free Book



Scan to Download



Listen It



Read, Share, Empower

Finish Your Reading Challenge, Donate Books to African Children.

The Concept



×



×



This book donation activity is rolling out together with Books For Africa. We release this project because we share the same belief as BFA: For many children in Africa, the gift of books truly is a gift of hope.

The Rule



Earn 100 points



Redeem a book



Donate to Africa

Your learning not only brings knowledge but also allows you to earn points for charitable causes! For every 100 points you earn, a book will be donated to Africa.

Free Trial with Bookey



Chapter 10 | 9 Concurrency: Practice| Q&A

1.Question

What is the consequence of improperly propagating a context in Go?

Answer:Improper context propagation can lead to subtle bugs, such as subfunctions not executing correctly. For example, if an HTTP handler sends a response and then dispatches a publish action using the request's context, the publish task can be interrupted if the context gets canceled, which might occur immediately after the response is sent.

2.Question

How can we prevent goroutine leaks in concurrent programming?

Answer:To prevent goroutine leaks, always ensure that you have a clear plan for stopping a goroutine. This might involve using context cancellation or implementing exit conditions, especially when you know the lifetime of the goroutines should correlate with the application lifecycle.

More Free Book



Scan to Download



Listen It

3.Question

What common mistake occurs with goroutines and loop variables?

Answer:A common mistake is that goroutines may capture loop variables incorrectly due to closures in Go, leading to non-deterministic behavior. To fix this, you can create a new local variable inside the loop to capture the current value or pass loop variables as arguments to the function.

4.Question

Why might using a buffered channel lead to unexpected behavior?

Answer:Buffered channels do not provide strong synchronization guarantees. As a result, if you're relying on the order of operations, there may be obscured deadlocks. Using an unbuffered channel can ensure that sender and receiver are synchronized properly.

5.Question

What are notification channels, and how should they be implemented?

Answer:Notification channels should be implemented using a

More Free Book



Scan to Download



Listen It

channel of empty structs (`chan struct{}`). This type utilizes no memory and indicates that only the fact of receiving a message is important, not its content.

6.Question

What is the danger of copying sync types in Go?

Answer: Copying sync types such as `sync.Mutex` or `sync.WaitGroup` can lead to significant problems, such as data races and unexpected behavior, since these types maintain internal states. Always pass them by reference to avoid unintended shared states.

7.Question

How does the `errgroup` package simplify handling multiple goroutines?

Answer: The `errgroup` package allows for easier synchronization and error handling for multiple goroutines. It provides a single way to manage the lifecycle of goroutines and automatically propagates errors from them without needing to explicitly manage shared state or synchronization.

8.Question

What principle governs the use of the `select` statement

More Free Book



Scan to Download



Listen It

with multiple channels?

Answer: When using select with multiple channels, if more than one operation is ready, Go will randomly choose one of them to proceed. This non-deterministic behavior means you cannot rely on the order of cases in select for execution.

9.Question

What should be remembered about using mutex locks with slices and maps?

Answer: Care must be taken to protect accesses to slices and maps with mutexes. Failing to do so can result in data races. It's important to lock the mutex during both read and write operations on shared slices or maps.

10.Question

Why are nil channels helpful in Go?

Answer: Nil channels can be useful for signaling across goroutines because a read from a nil channel blocks indefinitely, allowing for simpler state management without busy loop conditions. They can serve as a mechanism to remove case conditions from select statements.

More Free Book



Scan to Download



Listen It

Chapter 11 | 10 The standard library| Q&A

1.Question

What is the importance of using the `time.Duration`` API when working with time-related functions in Go?

Answer:Using the `time.Duration`` API ensures that you provide durations in units that the Go standard library expects, preventing common errors due to confusion with numeric types from other languages. For example, if you mistakenly provide an integer directly (like 1000), it gets interpreted as 1000 nanoseconds, leading to unexpected behaviors like a ticker firing every microsecond instead of every second.

2.Question

How can using `time.After`` repeatedly in loops lead to memory leaks?

Answer:`time.After`` allocates resources for a channel that is only released when the timer expires. If called in a loop without proper handling (like resetting), these allocations can

More Free Book



Scan to Download



Listen It

cumulatively consume a significant amount of memory, potentially leading to inefficient memory use, especially under high-load scenarios.

3.Question

What happens to an embedded field when marshaling a struct in Go containing ``time.Time``?

Answer:When you embed a ``time.Time`` field in a struct, the struct also implements the ``json.Marshaler`` interface because ``time.Time`` does. This means the default marshaling behavior of the embedded field is used, often leading to the omission of other fields (like an ID) during serialization unless specifically handled.

4.Question

What should developers do to avoid unexpected results when marshaling and unmarshaling ``time.Time`` values?

Answer:Developers should use the ``Equal`` method to compare ``time.Time`` values instead of the ``==`` operator, as the latter also compares monotonic clocks which may lead to unexpected results due to differences in how ``time.Time`` is

More Free Book



Scan to Download



Listen It

constructed during marshaling and unmarshaling.

5.Question

What is a common mistake related to handling numeric values in JSON unmarshaling?

Answer:When unmarshaling into a ``map[string]any``, numeric values from JSON are converted to ``float64`` types, which can lead to type-related errors if you assume they remain integers. Always be cautious with type expectations when using ``any``.

6.Question

Why is it crucial to close transient resources in Go applications?

Answer:Closing transient resources like database connections, HTTP response bodies, or file handles prevents memory leaks and keeps system resources free for reuse. Failing to do so can lead to exhaustion of available connections or memory that may degrade performance.

7.Question

What errors might arise when handling SQL queries and how can they be mitigated?

More Free Book



Scan to Download



Listen It

Answer:Common errors include mishandling null values and forgetting to check for errors after iterating rows. Using wrapper types like ``sql.NullString`` for nullable columns and checking ``rows.Err()`` after iterations ensures stable error handling.

8.Question

What are the risks of using the default HTTP client and server in production?

Answer:Default HTTP clients and servers in Go lack important configurations such as timeouts, which can lead to indefinitely hanging requests or resources being exhausted by unresponsive clients. Thus, it's best practice to create and configure custom clients and servers with appropriate settings.

9.Question

How should an HTTP handler manage responses to prevent executing unwanted code?

Answer:An HTTP handler should always return immediately after sending an error response using ``http.Error`` to prevent



further code from executing. Failing to do this could lead to inconsistent responses and unexpected behaviors in the application.

10.Question

What are the main configuration parameters for database connection pools, and why are they important?

Answer:Key parameters include maximum open connections, maximum idle connections, idle connection timeout, and maximum connection lifetime. Configuring these ensures optimal resource usage and can significantly impact performance, especially under load.

Chapter 12 | 11 Testing| Q&A

1.Question

What is the significance of categorizing tests in Go?

Answer:Categorizing tests helps to save time and maintain accuracy in testing the software. By using the testing pyramid, Go allows developers to run unit tests frequently, ensuring that the base of their testing structure is robust and efficient. This

More Free Book



Scan to Download



Listen It

categorization can include build tags that specify the type of tests (e.g., integration vs. unit), helping to better manage test execution based on project stage.

2.Question

Why is it important to enable the -race flag when testing?

Answer:Enabling the -race flag is crucial as it helps in detecting data races in concurrent applications. A data race occurs when two goroutines access the same variable simultaneously, with at least one of them modifying it. Using the race detector identifies potential issues that could lead to bugs, ensuring more stable and reliable concurrent code.

3.Question

How can using the -parallel flag improve test performance?

Answer:The -parallel flag allows specified tests to run concurrently, which can significantly reduce overall test execution time, especially for a large suite of long-running tests. It facilitates optimal resource utilization by executing multiple tests at the same time, rather than sequentially.

More Free Book



Scan to Download



Listen It

4.Question

What is a table-driven test and why should it be used in Go?

Answer:A table-driven test is a way to organize tests that share similar patterns, reducing boilerplate code and making tests easier to read and maintain. By using maps or structs to define input and expected output for various test cases, it centralizes the test logic, preventing duplication and facilitating easier updates.

5.Question

How can one avoid flakiness in tests when dealing with sleep?

Answer:To avoid flakiness caused by `time.Sleep``, methods such as implementing retries (to periodically assert a condition) or using channels for synchronization should be utilized. These practices make tests more deterministic and reliable compared to simply waiting a fixed duration.

6.Question

What strategies can be employed to handle testing code that uses the time API efficiently?

More Free Book



Scan to Download



Listen It

Answer: Strategies include dependency injection for the time function that can be mocked in tests or redesigning functions to accept time as an argument to avoid reliance on real-time calls. This leads to more predictable and robust tests by eliminating variability due to the external time API.

7.Question

What are the benefits of using the `httptest` package in Go?

Answer: The `httptest` package simplifies testing of HTTP servers and clients by providing utilities to create mock servers and requests without needing to spin up real servers. This facilitates faster and more efficient tests by isolating HTTP interactions and focusing solely on the functionality of handlers and clients.

8.Question

Why should benchmarks be handled carefully in Go?

Answer: Benchmarks must be handled carefully due to potential pitfalls such as timing inaccuracies, compiler optimizations that might lead to misleading results, and the

More Free Book



Scan to Download



Listen It

observer effect that can skew metrics. Ensuring the benchmark measures effectively by using proper timer methods and handling setup correctly is essential for obtaining reliable performance data.

9.Question

What should one consider regarding code coverage in tests?

Answer:While achieving high code coverage is beneficial for identifying untested paths in the code, it is important to focus on the quality of tests rather than just coverage percentage. Tools like `-coverprofile`` help visualize coverage, but one must interpret results critically to ensure meaningful testing rather than memorizing arbitrary coverage goals.

10.Question

How does using a separate `_test` package for tests help in Go?

Answer:Using a separate `_test` package enforces a testing philosophy that focuses on testing the public interface of the code rather than its implementation details. This leads to

More Free Book



Scan to Download



Listen It

more robust tests that can seamlessly adapt to code changes, as they verify behaviors through the API that clients would use, rather than relying on internal state.

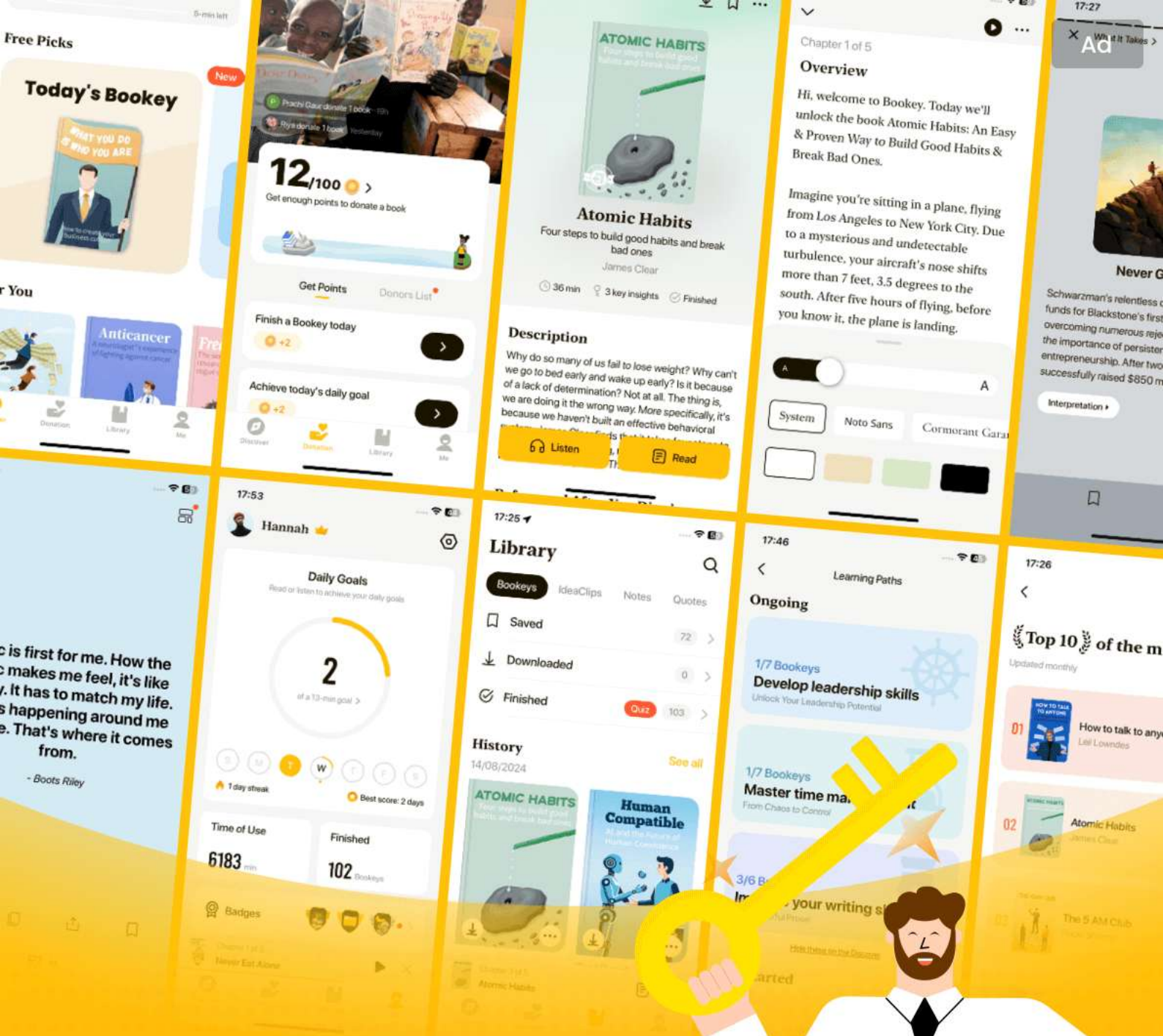
More Free Book



Scan to Download



Listen It



World's best ideas unlock your potential

Free Trial with Bookey



Scan to download



Chapter 13 | 12 Optimizations| Q&A

1.Question

What is mechanical sympathy and why is it important in optimization for Go applications?

Answer: Mechanical sympathy refers to understanding how a system, like a CPU, is designed to work and optimizing code accordingly. It is important in Go applications because understanding hardware fundamentals, like CPU caches and memory architecture, allows developers to write more efficient code that better utilizes CPU resources, resulting in improved performance without sacrificing readability.

2.Question

What are CPU caches and how do they influence Go application performance?

Answer: CPU caches are small, fast types of memory located closer to a processor core, designed to speed up data access. In a Go application, efficient data access patterns that favor

More Free Book



Scan to Download



Listen It

the use of CPU caches can significantly enhance performance by reducing the time spent accessing slower main memory. For example, organizing data with good spatial locality allows for the use of cache lines effectively, enhancing throughput.

3.Question

How does 'looping over slices' affect cache hits and misses, and what can be done to optimize it?

Answer: Accessing elements in a Go slice efficiently utilizes CPU caches due to spatial and temporal locality. When iterating over a slice, accessing elements consecutively results in fewer cache misses. To optimize it, you can structure loops to access multiple elements (like iterating every 2 or 8 elements) while keeping in mind that the benefit diminishes beyond a certain point due to the structure of cache lines.

4.Question

What is false sharing and how can it degrade performance in concurrent Go applications?

More Free Book



Scan to Download



Listen It

Answer:False sharing occurs in concurrent applications when multiple goroutines modify variables that are packed together in memory, leading to unnecessary cache invalidations. This degrades performance as it forces the CPU to synchronize cache lines, wasting processing time. To avoid false sharing, developers can pad structures to ensure that related variables are placed in separate cache lines.

5.Question

What is instruction-level parallelism (ILP) and how can understanding it help optimize Go code?

Answer:Instruction-level parallelism (ILP) allows a CPU to execute multiple instructions simultaneously. Understanding ILP can help developers design code that minimizes hazards (like data hazards) and allows for greater parallel instruction execution. This could involve rearranging instructions or using temporary variables to maintain the state across operations, which ultimately leads to more efficient code.

6.Question

How does data alignment impact performance, and how can it be optimized in Go?

More Free Book



Scan to Download



Listen It

Answer: Data alignment ensures that variables are stored at memory addresses that are multiples of their sizes to prevent the CPU from having to access multiple memory locations. Poor alignment can lead to wasted memory and increased access times. In Go, optimizing struct layouts by ordering fields from largest to smallest can reduce padding and improve memory access patterns.

7.Question

What are the differences between stack and heap memory allocations and their impact on Go applications?

Answer: Stack memory is faster and automatically managed, while heap memory is slower and requires garbage collection (GC). In Go, variables allocated on the stack are deallocated automatically at the end of a function, while heap allocations increase GC pressure, potentially causing delays.

Understanding these differences is crucial for optimizing memory usage in Go applications.

8.Question

How does garbage collection (GC) work in Go, and what strategies can be employed to optimize performance?

More Free Book



Scan to Download



Listen It

Answer:Garbage collection in Go uses a mark-and-sweep algorithm that identifies and reclaims memory that is no longer in use. It runs concurrently but can cause 'stop-the-world' pauses. Developers can tune the GC with the GOGC variable, adjusting how aggressively memory is collected to balance performance and memory usage. Understanding and optimizing GC behavior can help maintain application responsiveness, especially during high-load scenarios.

9.Question

What are the implications of running Go applications in Docker and Kubernetes, particularly concerning CPU allocation?

Answer:Running Go applications in Docker and Kubernetes can lead to issues like CPU throttling if GOMAXPROCS is set without considering the resource limits configured in containers. This can result in longer execution times due to limitations on available CPU cores. Developers should consider using libraries like automaxprocs that adjust

More Free Book



Scan to Download



Listen It

GOMAXPROCS according to container limits to prevent such throttling.

10.Question

How can profiling and tracing improve the performance of Go applications?

Answer: Profiling and tracing provide insights into application performance, helping developers identify bottlenecks, memory leaks, and contention issues. Tools like pprof allow developers to visualize CPU and memory usage, enabling targeted optimizations. These diagnostics are critical for performance tuning and understanding the execution patterns of applications.

More Free Book



Scan to Download



Listen It

Ad



Scan to Download



Try Bookey App to read 1000+ summary of world best books

Unlock **1000+** Titles, **80+** Topics

New titles added every week

Brand



Leadership & Collaboration



Time Management



Relationship & Communication



Business Strategy



Creativity



Public



Money & Investing



Know Yourself



Positive Psychology

Entrepreneurship



World History



Parent-Child Communication

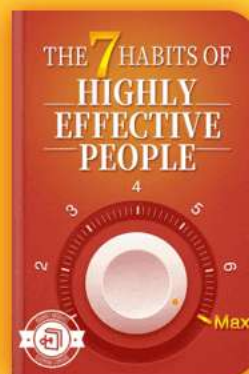
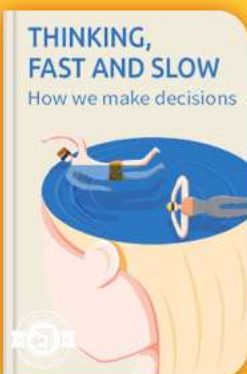


Self-care



Mind & Spirituality

Insights of world best books



Free Trial with Bookey



100 Go Mistakes and How to Avoid Them Quiz and Test

[Check the Correct Answer on Bookey Website](#)

Chapter 1 | 100 Go Mistakes| Quiz and Test

1. Teiva Harsanyi began his journey with Go as the primary language in 2019.
2. The book '100 Go Mistakes and How to Avoid Them' is primarily intended for beginners in Go.
3. The book contains 100 common mistakes and is structured into 12 chapters covering various Go programming topics.

Chapter 2 | 1 Go: Simple to learn but hard to master| Quiz and Test

1. Go programming language has a steep learning curve despite its simplicity.
2. Learning from mistakes is a critical component in becoming proficient in Go.
3. Go's syntax is complex and difficult to understand for beginners.

Chapter 3 | 2 Code and project organization| Quiz

More Free Book



Scan to Download



[Listen It](#)

and Test

1. Variable shadowing can lead to unintended behavior and should be avoided by using temporary variables or direct assignments.
2. Overusing interfaces in Go is encouraged as it adds complexity and allows for better future-proofing of the code.
3. Using the empty interface (`any`) is recommended in Go as it provides more flexibility in type usage.

More Free Book



Scan to Download



Listen It

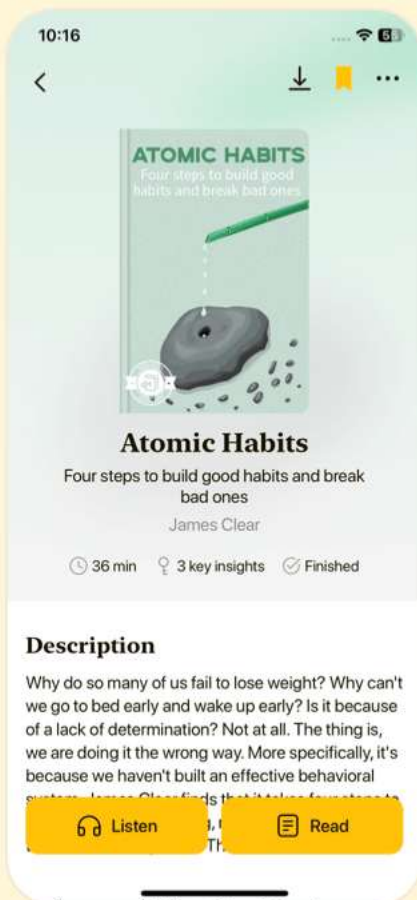


Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Scan to Download



Chapter 4 | 3 Data types| Quiz and Test

1. Octal integer literals in Go are indicated by a leading zero (0).
2. Go's maps automatically shrink in size when elements are cleared, reducing memory consumption.
3. Using the `len(slice) == 0` method is the idiomatic way to check if a slice is empty in Go.

Chapter 5 | 4 Control structures| Quiz and Test

1. A range loop in Go creates copies of elements during iteration, meaning any updates made to the element inside the loop will not affect the original element.
2. In Go, when using range loops, modifications to a map during iteration will always produce predictable results.
3. The break statement in Go exits all levels of control structures, including outer loops, without the need for labels.

Chapter 6 | 5 Strings| Quiz and Test

1. A rune in Go represents a Unicode code point and



is important for string operations.

2. Using the ``+=`` operator for string concatenation is the most efficient method in Go.

3. String literals in Go are encoded in UTF-16.

More Free Book



Scan to Download



Listen It

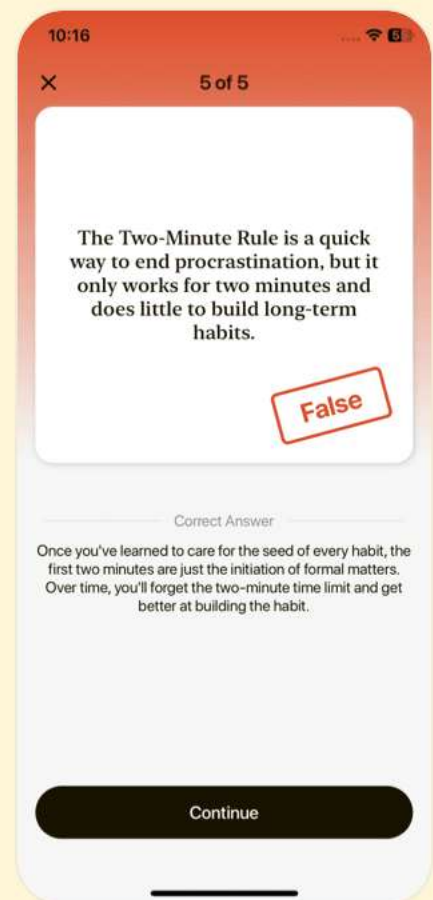
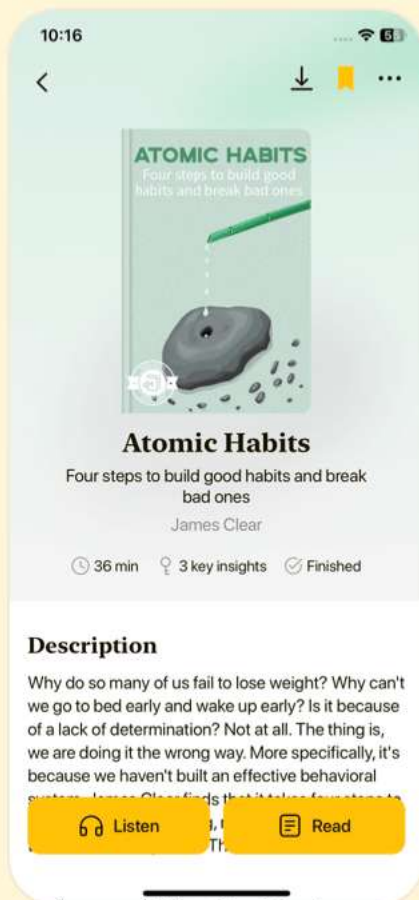


Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Scan to Download



Chapter 7 | 6 Functions and methods| Quiz and Test

- 1.Choosing between value and pointer receivers is not crucial in Go.
- 2.Named result parameters can enhance clarity in function signatures when used properly.
- 3.You should always return a nil receiver when dealing with interfaces, as it avoids returning unexpected values.

Chapter 8 | 7 Error management| Quiz and Test

- 1.In Go, errors are treated as regular return values rather than using try/catch mechanisms.
- 2.Developers should use panic frequently to interrupt the normal flow of a program.
- 3.Developers must use `errors.As` for checking specific error types instead of type assertion to handle wrapped errors correctly.

Chapter 9 | 8 Concurrency: Foundations| Quiz and Test

- 1.Concurrency and parallelism are the same concepts in Go programming.

More Free Book



Scan to Download



Listen It

2. Using channels is preferred over mutexes when transferring ownership of data between goroutines.
3. The Go memory model does not play a significant role in preventing data races in concurrent programming.

More Free Book



Scan to Download



Listen It

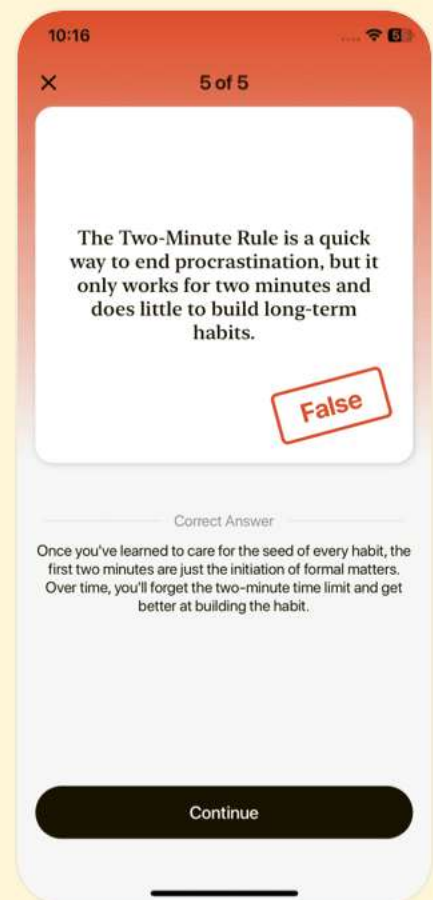
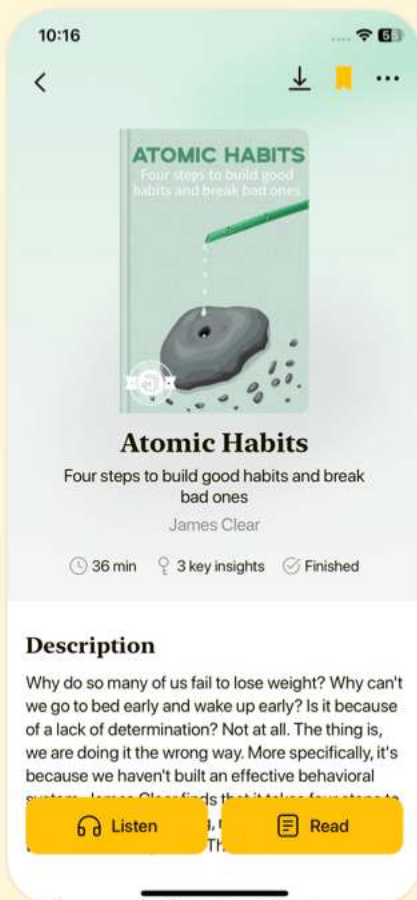


Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Scan to Download



Chapter 10 | 9 Concurrency: Practice| Quiz and Test

1. Goroutines can lead to memory leaks if they are started without knowing when to stop them.
2. It is safe to copy mutexes in Go applications without leading to data races.
3. Using 'sync.WaitGroup' requires the Add method to be called after launching goroutines for correct counting.

Chapter 11 | 10 The standard library| Quiz and Test

1. When using `time.Duration`, developers often mistakenly provide durations in milliseconds instead of nanoseconds.
2. The `sql.Open` function immediately establishes a connection to the database without needing to use the `Ping` function.
3. Using the default HTTP client and server in a production environment is advisable because they are configured with essential timeout settings.

Chapter 12 | 11 Testing| Quiz and Test

1. Testing is not important for a project's lifecycle.

More Free Book



Scan to Download



Listen It

2.Using time.Sleep in unit tests can lead to more robust testing.

3.The race detector tool in Go should always be enabled to identify data races during testing.

More Free Book



Scan to Download



Listen It

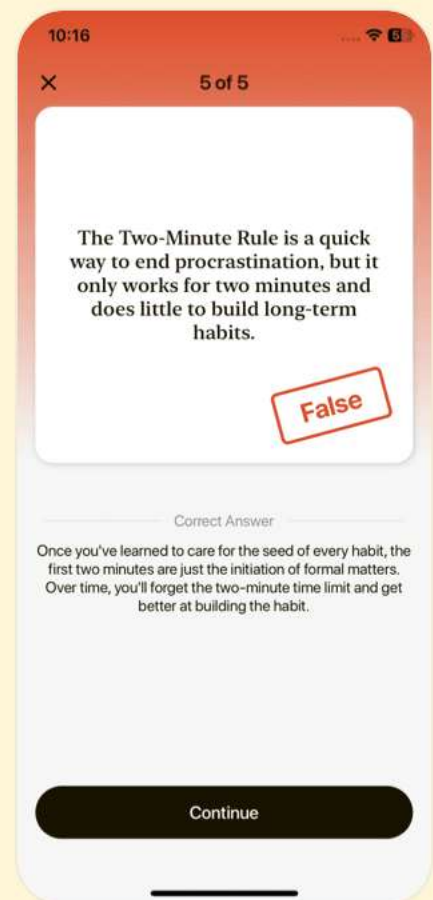
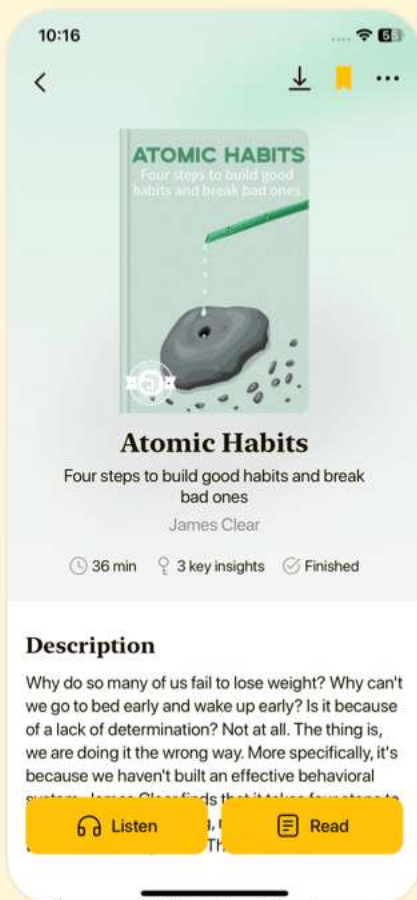


Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Scan to Download



Chapter 13 | 12 Optimizations| Quiz and Test

1. Readability and clarity of code should be emphasized before performance in Go applications.
2. Proper data alignment is not critical for optimal memory access in Go programming.
3. Using inline functions can increase call overhead and hinder compiler optimizations.

More Free Book



Scan to Download



Listen It



Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Scan to Download

